

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



Webová aplikace pro správu testů pro střední
školy

BAKALÁŘSKÁ PRÁCE

Vypracoval: Adam Jírotka

Vedoucí práce: Ing. Jan Burle, Ph.D.

Studijní program: Aplikovaná informatika

ÚSTÍ NAD LABEM 2026

UNIVERZITA JANA EVANGELISTY PURKYNĚ V ÚSTÍ NAD LABEM

Přírodovědecká fakulta

Akademický rok: 2024/2025

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

Jméno a příjmení: Adam JIROTKA
Osobní číslo: F21253
Studijní program: B0613P140005 Aplikovaná informatika
Téma práce: Webová aplikace pro správu testů pro střední školy
Zadávající katedra: Katedra informatiky

Zásady pro vypracování

Zpracování testů je nezbytnou částí digitalizace výuky, která zlepšuje komunikaci a interakci mezi učiteli a žáky. Stávající řešení postrádají intuitivní ovládání a kvalitní uživatelskou zkušenost (UX).

Hlavním cílem práce je navrhnout a implementovat uživatelské rozhraní (frontend) webové aplikace pro efektivní správu a tvorbu testů, které bude klást důraz na intuitivní ovládání a kvalitní uživatelskou zkušenost (UX/UI design). Práce je součástí projektu pro gymnázia v Aši, Chebu a Jihlavě. Jejím výsledkem bude návrh frontendu podle požadavků zainteresovaných osob (učitelé gymnázií převážně přírodovědných předmětů). Aplikace bude komunikovat s backendem pomocí webového API (backend je implementován souběžně v rámci projektu). Primární funkce budou zahrnovat:

- oddělené uživatelské rozhraní pro učitele a žáky
- správa testů (vytváření, úprava, pohled žáka, vkládání obrázků)
- vyplňování testů a zpětná vazba (hodnocení)
- různé typy úloh na výběr
- nastavení počtu pokusů na zvládnutí testu
- možnost nastavit konkrétní délku trvání testu v rámci celé doby, kdy bude přístupný

Aplikace bude oproti konkurenčním řešením (např. Moodle) klást důraz na intuitivní design. Dále bude obsahovat push notifikace z backendu v reálném čase. Návrh aplikace bude navíc optimalizován pro portaci na mobilní platformy.

Osnova:

Teoretická část

- didaktické využití testů ve vzdělávání
- návrh a optimalizace UI a UX
- charakteristika použitých webových technologií

Praktická část

- analýza existujících řešení
- analýza požadavků zadavatelů
- návrh UI řešení
- tvorba UX nástrojů
- optimalizace UX
- implementace frontendu

Forma zpracování bakalářské práce: tištěná/elektronická

Seznam doporučené literatury:

1. ALESSANDRIA, Simone. *Flutter Cookbook – Second Edition: 100+ real-world recipes to build cross-platform applications with Flutter 3.x powered by Dart 3 (alpha)*. Packt Publishing, 2023. ISBN 978-1803245430.
2. Apigee. *Web API Design: The Missing Link* [online]. Google Cloud, 2018. [cit. 2024-11-14]. Dostupné z: <https://cloud.google.com/files/apigee/apigee-web-api-design-the-missing-link-ebook.pdf>
3. Flutter Dev Team. *Flutter Documentation* [online]. Google, 2024. [cit. 2024-11-14]. Dostupné z: <https://docs.flutter.dev>
4. GOTHELF, Jeff a SEIDEN, Josh. *Lean UX: Designing Great Products with Agile Teams*. 3rd Edition. O'Reilly Media, 2021. ISBN 978-1098116309.
5. KRUG, Steve. *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*. 3rd Edition. New Riders, 2013. ISBN 978-0321965516.
6. KYRIACOU, Chris. *Essential Teaching Skills*. 5th Edition. Oxford University Press, 2018. ISBN 978-0198423300.
7. PEREA, Pablo, a GINER, Pau. *UX Design for Mobile: Design apps that deliver impressive mobile experiences*. 1st Edition. Packt Publishing, 2017. ISBN 978-1787283428.

Vedoucí bakalářské práce: Ing. Jan Burle, PhD
Katedra informatiky

Datum zadání bakalářské práce: 3. prosince 2024
Termín odevzdání bakalářské práce: 3. července 2026

RNDr. Jiří
Škvor, Ph.D. L.S. Digitálně podepsal
RNDr. Jiří Škvor, Ph.D.
Datum: 2026.07.02
22:50:25 +02'00'

doc. RNDr. Michal Varady, Ph.D.
děkan

RNDr. Jiří Škvor, Ph.D.
vedoucí katedry

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 3. července 2026

Podpis:

Děkuji vedoucímu práce Ing. Janu Burlemu, Ph.D.
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Abstrakt:

Tato bakalářská práce se zabývá návrhem a implementací webové aplikace pro správu testů určené pro střední školy. Hlavním cílem bylo vytvořit moderní uživatelské rozhraní, které zjednodušuje tvorbu a správu testů s důrazem na intuitivní ovládání a kvalitní uživatelskou zkušenost (UX/UI). Aplikace umožňuje oddělené prostředí pro učitele a žáky, tvorbu různých typů úloh, časová omezení a zpětnou vazbu po vyplnění testu. Komunikuje s backendem přes webové API a je navržena tak, aby byla snadno přenositelná na mobilní platformy. Součástí návrhu bylo také zohlednění požadavků učitelů z gymnázií v Aši, Chebu a Jihlavě. Výsledná aplikace se odlišuje od stávajících řešení důrazem na jednoduchost, efektivitu a uživatelskou přívětivost.

Klíčová slova: webová aplikace, školní testy, UX/UI design, střední školy, frontend

Abstract:

This bachelor's thesis focuses on the design and implementation of a web application for managing tests in secondary schools. The main objective was to create a modern user interface that simplifies test creation and management, with an emphasis on intuitive control and high-quality user experience (UX/UI). The application provides separate interfaces for teachers and students, supports various types of questions, time limitations, and feedback after test completion. It communicates with the backend through a web API and is designed for easy portability to mobile platforms. The development process was guided by the requirements of teachers from grammar schools in Aš, Cheb, and Jihlava. The resulting application differs from existing solutions by prioritizing simplicity, efficiency, and user-friendliness.

Keywords: web application, school tests, UX/UI design, secondary schools, frontend

Obsah

Úvod	13
1. Didaktické využití testů ve vzdělávání	15
1.1. Vymezení didaktického testu a jeho role ve výuce	15
1.2. Funkce testování: Od ověřování k učení (The Testing Effect)	16
1.3. Význam zpětné vazby v digitálním prostředí	16
1.4. Specifika počítačového testování (E-Assessment)	17
1.5. Vliv uživatelského rozhraní na validitu testu	17
1.6. Typologie testových úloh a jejich didaktický potenciál	18
1.6.1. Uzavřené úlohy	18
1.6.2. Otevřené úlohy	18
1.6.3. Integrace obou typů úloh v aplikaci	19
1.7. Specifika mobilního testování	19
2. Teorie návrhu a optimalizace UX/UI	21
2.1. Rozdíl mezi UI a UX a jejich role ve vzdělávacích aplikacích	21
2.2. Principy použitelnosti (Nielsenovy heuristiky)	22
2.3. Psychologie v designu	23
2.3.1. Hickův zákon a minimalismus	23
2.3.2. Fittsův zákon a optimalizace dotykového ovládání	23
2.4. Vizualní design a designové systémy	24
2.5. Responsivní design a přístupnost	25
3. Charakteristika použitých webových technologií	27
3.1. Architektura moderních webových aplikací (Client-Server)	27
3.2. Framework Flutter a multiplatformní vývoj	28
3.3. Programovací jazyk Dart	29
3.4. Komunikace s backendem (REST API a JSON)	30
3.5. Správa stavu aplikace	31
4. Analýza existujících řešení	35
4.1. Globální systémy pro řízení výuky (LMS)	35
4.2. Lokální školní informační systémy (ŠIS)	36
4.3. Komerční vzdělávací platformy (SaaS)	37

4.4.	Architektonická a technologická východiska pro návrh nového frontendu	38
5.	Analýza požadavků a návrh uživatelského rozhraní	39
5.1.	Identifikace cílových skupin a oddělená rozhraní	39
5.2.	Rozhraní pedagoga: Správa testů a tvorba obsahu	39
5.3.	Rozhraní žáka: Testovací proces a mobilní optimalizace	40
5.4.	Notifikace a formativní zpětná vazba	41
5.5.	Shrnutí funkčních a nefunkčních požadavků	46
6.	Proces návrhu a využití UX/UI nástrojů	47
6.1.	Definice cílových uživatelů	47
6.1.1.	Persona 1: Učitel přírodovědných předmětů	47
6.1.2.	Persona 2: Středoškolský student	48
6.2.	Nástroje pro návrh a prototypování	48
6.3.	Tvorba design systému (Design System & UI Kits)	49
6.4.	Mapování uživatelských cest a interakce	51
6.5.	Optimalizace pro použitelnost	52
7.	Implementace frontendu	55
7.1.	Softwarová architektura a struktura repozitáře	55
7.2.	Zpracování dat a komunikace s Backendem (API Client)	56
7.3.	Správa stavu (State Management) a Riverpod	57
7.4.	Implementace klíčových a komplexních modulů	58
7.5.	Routování a ochrana přístupu (GoRouter)	59
8.	Diskuse a výsledky	61
9.	Závěr	65
	Seznam použitých zdrojů	65
	Seznam obrázků	68
A.	Externí přílohy	71

Úvod

Digitalizace vzdělávacího procesu představuje v současné pedagogice jedno z nejvýznamnějších témat. Přejít od tradičních papírových metod směrem k počítačově podporovanému testování s sebou přináší nesporné výhody v podobě automatizace hodnocení, okamžité zpětné vazby a možnosti začlenění interaktivních multimediálních prvků. I přes tyto technologické přínosy však integrace elektronického testování do běžné středoškolské praxe často naráží na bariéru v podobě samotného uživatelského rozhraní aplikací.

Mnoho stávajících platforem pro řízení výuky, které školy využívají, se vyznačuje vysokou vizuální a funkční komplexitou. Složitě ovládání klade na pedagogy vysoké časové nároky při přípravě výukových materiálů, zatímco u studentů vyvolává tzv. vnější kognitivní zátěž a technologickou úzkost. Pokud žák během testu musí plýtvat mentální energií na orientaci v nepřehledném systému, klesá validita samotného didaktického měření, neboť test přestává zjišťovat reálné znalosti a začíná měřit schopnost ovládat aplikaci.

Hlavní motivací pro vznik této bakalářské práce je snaha tento problém eliminovat. Cílem práce je navrhnout a implementovat uživatelské rozhraní (frontend) nové webové aplikace pro efektivní správu a tvorbu testů, která bude určena primárně pro střední školy. Práce vzniká jako součást širšího projektu elektronizace výuky pro gymnázia v Aši, Chebu a Jihlavě, přičemž těžištěm návrhu je absolutní důraz na intuitivní ovládání, minimalismus a propracovanou uživatelskou zkušenost (UX/UI design). Aplikace si klade za cíl poskytnout oddělené, na míru šité pracovní prostředí jak pro učitele, tak pro žáky, s plnou optimalizací pro mobilní zařízení.

Text práce je strukturován do sedmi na sebe logicky navazujících kapitol. První tři kapitoly představují teoretický rámec; zabývají se didaktickým využitím testů ve vzdělávání, principy interakčního designu a kognitivní psychologie a charakteristikou moderních webových technologií, v čele s frameworkem Flutter. Čtvrtá kapitola analyzuje existující řešení na trhu a definuje jejich slabá místa. Pátá a šestá kapitola tvoří stěžejní část práce věnovanou samotnému procesu návrhu uživatelského rozhraní – od specifikace požadavků, přes definici uživatelských person, až po konkrétní aplikaci psychologických fenoménů (např. Hickova a Fittsova zákona) do finálního designu. Sedmá kapitola následně popisuje technologickou implementaci navrženého řešení, architekturu repozitáře a způsob asynchronní komunikace frontendu se serverovou částí.

1. Didaktické využití testů ve vzdělávání

1.1. Vymezení didaktického testu a jeho role ve výuce

V moderní pedagogice představuje didaktický test specifický nástroj, který se od běžné zkoušky odlišuje především svým systematickým pojetím a snahou o objektivizaci hodnocení. Podle definice Pavla Byčkovského [1] je didaktický test chápán jako nástroj systematického zjišťování výsledků výuky. Tato definice zdůrazňuje skutečnost, že test není nahodilou prověrkou, ale instrumentem, který je navrhován, ověřován a interpretován podle přesně stanovených pravidel.

Základní funkcí didaktického testu ve vyučovacím procesu není pouhá klasifikace žáků. Jeho role je v moderním pojetí mnohem širší a spočívá především v poskytování objektivní zpětné vazby – a to jak pro žáka, tak pro učitele. Testování slouží jako prostředek školního měření, které poskytuje kvantitativní popis výkonu žáka. Správně sestavený test eliminuje subjektivní vliv osobnosti hodnotitele a dává všem testovaným rovné šance, čímž naplňuje princip objektivity. Zejména průběžné (formativní) testy se v praxi často nepoužívají primárně k hodnocení žáků, ale k hodnocení úspěšnosti samotné výuky a k jejímu následnému optimálnímu řízení.

V pedagogické praxi se rozlišují dvě základní kategorie testů podle míry jejich přípravy:

- **Standardizované testy:** Jsou připravovány profesionálními institucemi (např. Cermat, Scio) a procházejí důkladným ověřováním na reprezentativních vzorcích (300–500 osob). Mají statisticky ověřené vlastnosti, jako je validita a reliabilita, a jsou doplněny testovou příručkou.
- **Nestandardizované (učitelské) testy:** Tyto testy si připravují učitelé sami pro potřeby konkrétní třídy. Ačkoliv u nich zpravidla neprobíhá komplexní statistické ověřování, jejich výhodou je vysoká flexibilita a schopnost reagovat na specifický obsah právě probraného učiva.

Právě podpora tvorby a správy učitelských testů je primárním cílem předkládané bakalářské práce. Tvorba kvalitního didaktického testu je časově i odborně náročná a stávající softwarová řešení často postrádají potřebnou intuitivnost, což pedagogy od systematického testování odrazuje. Implementace modulu pro tvorbu testů v navrhované webové aplikaci má za cíl tento proces usnadnit – od konstrukce úloh až po jejich automatizované vyhodnocení. Tím aplikace umožní učitelům vytvářet kvalitní didaktické nástroje, které neslouží pouze k hodnocení, ale i k aktivnímu učení, a zároveň budou v běžných školních podmínkách vykazovat vyšší míru spolehlivosti a platnosti.

1.2. Funkce testování: Od ověřování k učení (The Testing Effect)

Pro pochopení didaktické hodnoty aplikace je nutné rozlišovat mezi sumativním a formativním hodnocením. Zatímco **sumativní** hodnocení (hodnocení výsledků učení) tradičně slouží ke klasifikaci a shrnutí znalostí za určité období, **formativní** hodnocení (hodnocení pro učení) se zaměřuje na podporu vzdělávacího procesu v jeho průběhu. Formativní testy slouží jako zdroj informací pro učitele k úpravě instrukcí a pro žáka k identifikaci a odstranění nedostatků.[2]

Zásadní argument pro digitalizaci testování přináší kognitivní psychologie, konkrétně fenomén známý jako testovací efekt (The Testing Effect). Roediger a Karpicke [3] dokazují, že akt testování není pouze neutrálním měřením stavu paměti, ale procesem, který paměť aktivně mění. Vybavování si informací během testu vede k výrazně lepší dlouhodobé retenci učiva než pouhé opakované čtení studijních materiálů.

Mechanismy tohoto efektu lze rozdělit na:

- **Přímé efekty:** Samotný proces vybavení informace posiluje paměťovou stopu. Testování funguje jako tzv. „žádoucí obtížnost“ (desirable difficulty) – namáhavější vybavování vede k trvalejší znalostem.
- **Zprostředkované efekty:** Pravidelné testování motivuje k průběžné přípravě a poskytuje jasnou informaci o mezerách ve vědomostech.

Oproti tradičním systémům (např. Moodle), které často zdůrazňují ověřovací roli, se navrhované řešení orientuje i na procvičování učiva. Funkce „nastavení více pokusů“ transformuje test z nástroje zkoušení na nástroj učení. Umožnění opakovaného průchodu testem nutí žáka k opakovanému vybavování informací, což je v souladu s poznatky Roedigera a Karpickeho nejefektivnější cesta k hlubokému osvojení znalostí.[3]

1.3. Význam zpětné vazby v digitálním prostředí

Didaktický test bez následné reflexe má pro proces učení pouze omezený význam. Zpětná vazba je považována za jeden z nejsilnějších faktorů ovlivňujících úspěšnost učení, přičemž jejím cílem je redukovat nepoměr mezi aktuálním porozuměním žáka a výukovým cílem.

Podle modelu Hattieho a Timperleyové [4] musí efektivní zpětná vazba odpovídat na tři otázky: „Kam směřuji?“, „Jak si vedu?“ a „Kam dál?“. Pouhé sdělení výsledku „správně/špatně“ je nedostatečné; pro hlubší učení musí zpětná vazba cílit na procesní úroveň a pomáhat žákovi v seberegulaci. Zpětná vazba je nejúčinnější, když se zaměřuje na konkrétní chybné interpretace učiva, nikoliv pouze absenci znalosti.

Digitální prostředí nabízí v tomto ohledu zásadní výhodu oproti papírové formě: okamžitost. Zatímco u papírových testů vzniká prodleva způsobená opravováním, počítačové systémy umož-

ňují strojové vyhodnocení ihned po dokončení úlohy. To je klíčové ve fázi upevňování nových vědomostí, kdy okamžitá korekce zabraňuje fixaci chyb.

1.4. Specifika počítačového testování (E-Assessment)

Přechod od tradičních metod „tužka-papír“ k počítačem podporovanému testování (Computer-Based Testing, CBT) není jen technickou změnou, ale komplexním posunem v metodice hodnocení. Moderní e-assessment neznamena pouze digitalizaci statických úloh, ale zahrnuje interaktivní prvky a klade důraz na celkovou uživatelskou zkušenost.[5]

Výhody počítačového testování: Hlavním přínosem je efektivita. Automatizované skórování může snížit časovou náročnost hodnocení o 25–50 %, což učitelé uvolňuje ruce od rutinní administrativy a poskytuje mu okamžitá data o výkonu třídy.[1] Zásadní je také možnost integrace multimédií. V přírodovědných předmětech, pro které je aplikace především určena, je vizualizace klíčová. Digitální prostředí umožňuje vkládat barevná schémata či fotografie, což napomáhá pochopení jevů lépe než černobílý tisk.[6]

Rizika a implementace v aplikaci: E-assessment však přináší riziko tzv. „počítačové úzkosti“ [7]. Pokud žák bojuje s ovládáním aplikace, jeho výkon klesá, nehledě na jeho znalosti předmětu. Navrhovaná aplikace na toto riziko reaguje důrazem na minimalistický design a systém mikrointerakcí, které uživatele plynule provázejí testem. Cílem je vytvořit prostředí, které je natolik intuitivní, že technologie přestává být bariérou a stává se transparentním prostředkem hodnocení.

1.5. Vliv uživatelského rozhraní na validitu testu

Validita (platnost) je klíčovou vlastností didaktického testu – určuje, zda test skutečně měří to, co měřit má. V kontextu CBT se však do validity promítá nová proměnná: kvalita uživatelského rozhraní.

Teoretickým východiskem je zde teorie kognitivní zátěže. Žák má během testu omezenou kapacitu pracovní paměti. Pokud je rozhraní aplikace neintuitivní, způsobuje tzv. vnější kognitivní zátěž [5]. Žák musí plýtvat mentální energií na pochopení ovládání aplikace, místo aby ji věnoval řešení úlohy. V takovém případě test přestává měřit znalosti z daného učiva, ale měří digitální gramotnost a schopnost orientace v systému, což zásadně snižuje validitu měření.[1]

Jak uvádí Zaharias a Poylymenakou [7], použitelnost e-learningu je přímo spjata s jeho pedagogickou hodnotou. Stávající řešení typu Moodle často trpí vysokou komplexitou, která pro studenty představuje zbytečnou bariéru. Cílem je tuto bariéru odstranit optimalizací frontendu. Prostředí je navrženo tak, aby minimalizovalo kognitivní zátěž spojenou s ovládáním, čímž se maximalizuje prostor pro prokázání skutečných vědomostí žáka.

1.6. Typologie testových úloh a jejich didaktický potenciál

Kvalita didaktického testu jako celku je primárně determinována kvalitou jednotlivých testových úloh, které tvoří jeho obsahovou náplň. Při návrhu webové aplikace pro správu testů je nezbytné implementovat různé typy úloh, neboť každý druh zjišťuje jinou úroveň osvojených vědomostí a dovedností. V pedagogické teorii se úlohy standardně člení na dvě základní skupiny: úlohy uzavřené a úlohy otevřené.[1]

1.6.1. Uzavřené úlohy

Uzavřené úlohy předkládají žákovi připravený soubor možností, ze kterých vybírá správnou odpověď, nebo provádí jinou operaci podle zadání.[1]

- **Dichotomické úlohy:** Žák volí mezi dvěma alternativami (např. ano/ne). Jejich konstrukce je jednoduchá, ale nesou vysoké riziko náhodného uhodnutí správné odpovědi.
- **Úlohy s výběrem odpovědi (multiple-choice):** Skládají se z kmene úlohy (otázka nebo problém) a nabídky odpovědí, která obsahuje správnou volbu a tzv. distraktory (nesprávné, ale pro neinformovaného žáka lákavé odpovědi). Za optimální se považuje použití čtyř alternativ, čímž se pravděpodobnost náhodného uhodnutí snižuje na 25 %.
- **Přiřazovací a uspořádací úlohy:** Vyžadují od žáka logické propojení pojmů ze dvou množin nebo jejich seřazení podle určitého kritéria (např. chronologicky). Tyto úlohy minimalizují hádání a jsou vhodné pro zjišťování porozumění souvislostem.

Z hlediska digitální implementace jsou uzavřené úlohy klíčové, neboť umožňují objektivní skórování strojem. Toto automatizované vyhodnocení poskytuje okamžitou zpětnou vazbu, šetří čas pedagoga a eliminuje subjektivní chybu při hodnocení.

1.6.2. Otevřené úlohy

Otevřené úlohy vyžadují, aby žák odpověď sám formuloval.

- **Úlohy se stručnou odpovědí:** Žák uvádí konkrétní údaj – slovo, číslo, symbol či vzorec. Didakticky jsou cenné tím, že prakticky vylučují hádání a vyžadují od žáka přesné vybavení znalostí z paměti.
- **Úlohy se širokou odpovědí (esej testy):** Žák tvoří rozsáhlejší text. Tyto úlohy jsou nezapustitelné pro zkoušení komplexních vědomostí a vyšších cílových úrovní podle Bloomovy taxonomie [1], jako je analýza, syntéza a hodnocení. Umožňují žákovi prokázat hloubku porozumění a schopnost řešit problémové situace.

Významnou didaktickou výhodou otevřených úloh je jejich vliv na dlouhodobou retenci učiva. Výzkumy v oblasti kognitivní psychologie potvrzují, že úlohy vyžadující vybavení z paměti vedou

k trvalejšímu zapamatování než úlohy vyžadující pouhé znovupoznání (výběr z možností). Akt aktivního vybavování informace [3] je totiž kognitivně náročnější a posiluje paměťovou stopu efektivněji.

1.6.3. Integrace obou typů úloh v aplikaci

Pro účely této bakalářské práce byla zvolena kombinace obou typů úloh. Ačkoliv je automatické vyhodnocení jedním z pilířů digitalizace, výhradní orientace na tento typ by mohla vést k měření pouze povrchních znalostí (memorování faktů). Implementace otevřených úloh se stručnou odpovědí v aplikaci umožňuje zachovat určitou míru automatizace a zároveň nutí žáka k hlubší kognitivní aktivitě.

Široké otevřené úlohy pak v systému slouží jako nástroj pro komplexní diagnostiku, kterou učitel vyhodnocuje manuálně, čímž je naplněn požadavek na pedagogickou hodnotu testování, která přesahuje pouhou mechanickou kontrolu. Vytvořené prostředí tak učiteli dává flexibilitu volit mezi efektivitou (uzavřené úlohy) a didaktickou hloubkou (otevřené úlohy) v závislosti na specifickém cíli výuky.

1.7. Specifika mobilního testování

V kontextu současné digitalizace vzdělávání představuje mobilní testování specifickou formu počítačového testování (CBT), která reflektuje globální trend dominance mobilních zařízení v přístupu k digitálnímu obsahu. Aktuální data ukazují, že více než polovina celosvětového webového provozu pochází z mobilních zařízení, což činí optimalizaci pro tyto platformy kritickým požadavkem pro úspěšnou implementaci jakéhokoli vzdělávacího softwaru.

Základním teoretickým i praktickým východiskem pro návrh frontendu je princip „mobile-first design“, který prioritizuje návrh rozhraní pro nejmenší obrazovky a dotykové interakce před následným škálováním pro desktopové verze. Tento přístup prokazatelně zvyšuje použitelnost a celkový výkon aplikace, což přímo ovlivňuje spokojenost uživatelů.[5]

Výzvy uživatelského rozhraní na mobilu

Rozhraní pro mobilní zařízení musí reflektovat specifické omezení hardwaru a odlišný způsob interakce uživatele. Mezi hlavní výzvy patří adaptace na různé velikosti displejů a zajištění designové konzistence napříč platformami. V mobilním prostředí je naprosto klíčový minimalistický design, který se zaměřuje pouze na esenciální prvky. Tím eliminuje vizuální balast (clutter), zvyšuje soustředění žáka na testovou úlohu a zásadně snižuje jeho kognitivní zátěž.[7] Vzhledem k omezenému prostoru obrazovky je žádoucí nahrazovat dlouhé a nepřehledné textové pasáže vhodnými grafickými prvky, schémata nebo obrázky, které jsou žáci schopni interpretovat mnohem rychleji a efektivněji.

Interakce a mikrointerakce

Uživatelská zkušenost na mobilních platformách je dále umocněna zahrnutím mikrointerakcí, jako jsou jemné animace, vizuální odezvy na stisknutí tlačítka a push notifikace v reálném čase. Tyto prvky slouží jako intuitivní vodítka, která uživatele provázejí procesem vyplňování testu a zvyšují responsivitu celého systému.

2. Teorie návrhu a optimalizace UX/UI

2.1. Rozdíl mezi UI a UX a jejich role ve vzdělávacích aplikacích

Pro úspěšný návrh a implementaci webové aplikace pro správu testů je nezbytné jasně vymezit a pochopit rozdíl mezi uživatelským rozhraním (User Interface, UI) a uživatelskou zkušeností (User Experience, UX). Ačkoliv spolu tyto aspekty v procesu vývoje úzce souvisejí, plní odlišné funkce.

Uživatelské rozhraní zahrnuje soubor metod a nástrojů, jejichž prostřednictvím uživatelé interagují se zařízením či aplikací.[8] V kontextu vizuálního návrhu se jedná o konkrétní „designový jazyk“, který definuje volbu barevných schémat, tvarů, vzorů, typografie a rozložení prvků (layout). UI se primárně zaměřuje na estetickou stránku a vizuální prezentaci. Jeho cílem je, aby produkt působil atraktivně, důvěryhodně a konzistentně.

Uživatelská zkušenost se naproti tomu zaměřuje na interakci a celkový prožitek uživatele při práci se systémem [8]. Nejde pouze o to, jak systém vypadá, ale jak funguje a jak efektivně umožňuje uživateli dosáhnout jeho cílů. UX design staví na heuristikách použitelnosti, které fungují jako obecná pravidla pro interakční design. Cílem je maximálně usnadnit interaktivitu mezi uživatelem a aplikací a zvýšit efektivitu uživatelské práce.

Role UX ve vzdělávání a minimalizace kognitivní zátěže

V oblasti vzdělávacích aplikací je kvalita UX kritickým faktorem. Pokud je rozhraní složité nebo neintuitivní, dochází k nežádoucímu zvyšování kognitivní zátěže studenta. Místo toho, aby student věnoval svou mentální kapacitu řešení testové úlohy, musí ji vynakládat na pochopení ovládání aplikace a orientaci v rozhraní.

Jakob Nielsen upozorňuje, že selhání v udržení konzistence nutí uživatele učit se nové principy, což přímo zvyšuje jejich kognitivní úsilí. Pro vzdělávací proces je podstatné minimalizovat nutnost pamatovat si ovládací prvky. Rozhraní by mělo podporovat spíše rozpoznávání (recognition) než vyžadovat aktivní vzpomínání (recall). Ideálním stavem je dosažení tzv. „neviditelného rozhraní“, kdy systém nevyžaduje žádné dodatečné vysvětlení a uživatel se orientuje zcela intuitivně.[9]

2.2. Principy použitelnosti (Nielsenovy heuristiky)

Při návrhu uživatelského rozhraní vzdělávací aplikace je nezbytné vycházet z ověřených pravidel interakčního designu. Jakob Nielsen definoval deset obecných principů, tzv. heuristik [9], které slouží jako široká pravidla pro zajištění použitelnosti. Pro kontext webové aplikace určené k testování žáků jsou klíčové především následující tři principy, které přímo ovlivňují plynulost a spolehlivost testovacího procesu:

Viditelnost stavu systému

Základním pravidlem je, že design by měl uživatele neustále informovat o tom, co se v systému děje, a to prostřednictvím vhodné zpětné vazby v přiměřeném čase. Pokud uživatelé znají aktuální stav systému, mohou lépe předvídat výsledky svých interakcí, což buduje důvěru v produkt. V kontextu školního testu je tento princip kritický pro snížení stresu studenta. Rozhraní musí jasně indikovat:

- **Časový limit:** Odpočet času musí být neustále viditelný, aby student věděl, kolik času mu zbývá na dokončení úloh.
- **Postup v testu:** Podobně jako indikátory „Zde se nacházíte“ na mapách, i aplikace musí studentovi vizualizovat, na které otázce se nachází a kolik jich ještě zbývá (např. formou ukazatele postupu nebo číslování „Otázka 5 z 20“).

Uživatelská kontrola a svoboda

Uživatelé často provádějí akce omylem a potřebují jasně označený „nouzový východ“, aby mohli opustit nechtěný stav bez zdlouhavého procesu. Podpora funkcí jako „zpět“ posiluje pocit svobody a jistoty při ovládání aplikace. V rámci aplikace pro správu testů se tento princip projevuje v možnosti navigace mezi otázkami. Pokud to didaktický záměr testu explicitně nezakazuje, žák by měl mít možnost vrátit se k předchozí otázce, zkontrolovat svou odpověď nebo ji změnit, aniž by se cítil v aplikaci uvězněn či frustrován nemožností opravy chybného kliknutí.

Prevence chyb

Ačkoliv jsou dobré chybové zprávy důležité, nejlepší design je takový, který pečlivě předchází vzniku problémů ještě předtím, než nastanou. Nielsen rozlišuje dva typy chyb [9]: přehmaty, které vznikají z nepozornosti, a omyly, které vycházejí z chybného mentálního modelu uživatele. Aplikace musí eliminovat riziko nechtěného odeslání nedokončeného testu (přehmat). Toho lze docílit zavedením potvrzovacích dialogů před finálním odesláním. Systém by měl před odesláním automaticky zkontrolovat, zda jsou vyplněny všechny povinné otázky, a v případě prázdných polí žáka upozornit, čímž aktivně brání vzniku chyby, která by mohla mít negativní dopad na hodnocení žáka.

2.3. Psychologie v designu

Při návrhu uživatelského rozhraní vzdělávací aplikace je nezbytné zohlednit kognitivní psychologii, která definuje limity lidského vnímání a motoriky. Cílem je minimalizovat mentální úsilí nutné k ovládání aplikace, aby se student mohl plně soustředit na obsah testu. K obhajobě rozložení prvků využíváme dva klíčové principy: Hickův zákon a Fittsův zákon.

2.3.1. Hickův zákon a minimalismus

Hickův zákon [10] popisuje vztah mezi časem potřebným k rozhodnutí a počtem dostupných možností. S rostoucím množstvím podnětů exponenciálně narůstá čas reakce uživatele. V kontextu navrhované aplikace je tento princip podpořen osmou heuristikou [9] Jakoba Nielsena „Estetický a minimalistický design“, která uvádí, že každá nadbytečná jednotka informace v rozhraní soutěží o pozornost s relevantními informacemi a snižuje jejich viditelnost.

Z tohoto důvodu bude rozhraní pro vyplňování testů navrženo striktně minimalisticky. Během aktivního testu budou z obrazovky odstraněny veškeré navigační prvky, patičky a menu, které by mohly studenta rozptylovat. Na obrazovce zůstane pouze znění otázky, možnosti odpovědí a nezbytné indikátory stavu (čas a progres). Tím se eliminuje riziko kognitivního přetížení a zrychlí se orientace v úloze.

2.3.2. Fittsův zákon a optimalizace dotykového ovládání

Fittsův zákon [11] je základním modelem v oblasti interakce člověk-počítač (Human-Computer Interaction, HCI), který slouží k predikci a měření lidského výkonu při cíleném pohybu. Zákon stanovuje, že čas potřebný k zasažení cíle je funkcí vzdálenosti k cíli (A - amplitude) a velikosti cíle (W - width). Matematicky je obtížnost úlohy (ID - Index of Difficulty) vyjádřena logaritmickou funkcí poměru vzdálenosti a šířky cíle:

$$ID = \log_2 \left(\frac{2A}{W} \right)^1 \quad (2.1)$$

Z rovnice vyplývá, že snížení obtížnosti interakce a její zrychlení lze dosáhnout zvětšením cíle (W) nebo zkrácením vzdálenosti k němu (A). Pro návrh mobilní verze aplikace je tento zákon kritický. Ačkoliv dotyková rozhraní mohou vykazovat vyšší propustnost než ovládání myši, narážejí na specifický problém přesnosti, známý jako „fat-finger problem“ [11], kdy malá velikost cílů vede k vysoké chybovosti.

Aplikace proto bude implementovat následující opatření vycházející z Fittsova zákona:

¹Vzorec vychází ze Shannonova teorému informační kapacity. Logaritmus o základu 2 vyjadřuje mentální a motorickou náročnost v *bitech* zpracované informace. Násobitel 2 u vzdálenosti A (amplitudy) jednak historicky reflektuje původní Fittsův experiment s opakovaným pohybem mezi dvěma terči, a zároveň matematicky zajišťuje, že výsledný Index obtížnosti (ID) nabývá vždy kladných hodnot.

- **Velikost ovládacích prvků:** Tlačítka pro kritické akce, jako je „Odeslat“ nebo „Další otázka“, budou dostatečně velká [8], aby se maximalizovala proměnná W a snížila se tak obtížnost zasažení cíle.
- **Umístění prvků:** Interaktivní prvky budou umístěny v dosahové zóně palce (ve spodní části obrazovky), čímž se minimalizuje vzdálenost A, kterou musí prst urazit.
- **Oddělení prvků:** Nahrazení textového vstupu výběrem z možností, kde to lze, a zajistit, aby navigační tlačítka byla dostatečně velká pro ovládání stylusem či prstem.

2.4. Vizuální design a designové systémy

Pro zajištění kvalitní vizuální stránky a konzistence uživatelského rozhraní bude aplikace využívat principy Material Designu [12]. Jedná se o moderní designový systém, který slouží jako teoretický a praktický rámec pro sjednocení vzhledu a chování aplikace napříč různými platformami, od mobilních telefonů po desktopy.

Designový systém a metafora reálného světa

Material Design definuje sadu pravidel, která pomáhají vývojářům vytvářet produkty s jednotným vizuálním stylem. Základní myšlenkou je snaha učinit uživatelské rozhraní přirozeným tím, že digitální prvky imitují chování objektů v reálném světě. V tomto systému existují prvky v 3D prostoru a mají svou váhu a pevnost, což znamená, že vstupy (dotyky) nemohou procházet skrz materiál. Pro vizuální hierarchii [12] je klíčové využití osy Z a stínů, které indikují, jak jsou jednotlivé vrstvy materiálu naskládány na sobě.

Barvy a zpětná vazba

Vizuální design není pouze estetickou záležitostí, ale plní i funkční roli při poskytování zpětné vazby.

- **Indikace chyb:** V souladu s heuristikami použitelnosti je nutné využívat tradiční vizuální prostředky pro chybová hlášení, jako je například tučný červený text, který uživatelům pomůže problém okamžitě rozpoznat a diagnostikovat.
- **Interaktivita:** Materiál reaguje na vstup uživatele okamžitou vizuální změnou. Například tlačítko při stisknutí může vystoupit do popředí nebo vrhnout stín, čímž dává uživateli jasnou odezvu, že akce byla systémem zaznamenána.

Typografie a čitelnost

Moderní designové systémy [12] kladou velký důraz na typografii a minimalismus. Cílem je odstranit nadbytečné grafické prvky a soustředit se primárně na obsah. Správné použití typografie

zajišťuje, že rozhraní je efektivní, vizuálně přitažlivé a informace jsou předávány v přirozeném a logickém pořadí, což usnadňuje čtení na digitálních obrazovkách.

2.5. Responsivní design a přístupnost

V kontextu digitalizace výuky na středních školách je nezbytné, aby navrhovaná webová aplikace nebyla pouze funkční na školních počítačích, ale plně použitelná i na mobilních zařízeních studentů. Tento požadavek vede k nutnosti implementace responsivního designu a dodržování zásad přístupnosti.

Responsivita a adaptace rozložení (Layout)

Při návrhu pro mobilní zařízení [8] narážíme na specifická omezení, zejména menší rozlišení obrazovky a absenci fyzické klávesnice či myši. Pro adaptaci rozložení ve vzdělávacích aplikacích je třeba využít následující funkcionality:

- **Navigace:** Zatímco na desktopu lze využít komplexní postranní menu či záložky, pro mobilní zařízení s vertikálním režimem zobrazení je vhodnější umístit navigační prvky do horní a spodní části obrazovky, mezi nimiž se nachází obsah.
- **Velikost prvků:** Je nutné zohlednit „fat-finger problem“ [11]. Tlačítka a interaktivní prvky musí být dostatečně velké, aby je bylo možné ovládat dotykem bez chybovosti, která je u malých cílů na dotykových displejích častá.

3. Charakteristika použitých webových technologií

3.1. Architektura moderních webových aplikací (Client-Server)

Vývoj moderního softwaru se opírá o architekturu typu Client-Server, která striktně odděluje prezentační vrstvu (frontend) od datové a logické vrstvy (backend). Toto rozdělení je nezbytné pro zajištění škálovatelnosti, bezpečnosti a udržitelnosti aplikace.

Frontend (Klientská část)

Frontend představuje uživatelské rozhraní, které běží v prohlížeči uživatele. Jeho úkolem je prezentovat data, reagovat na vstupy uživatele a zajišťovat interaktivitu. V moderním pojetí se nejedná pouze o statické zobrazení textu, ale o dynamické prostředí, které je často řízeno pokročilými JavaScriptovými frameworky.[13] V rámci tohoto projektu je frontend navržen tak, aby komunikoval s backendem prostřednictvím definovaného rozhraní, typicky REST API, což umožňuje výměnu dat ve standardizovaném formátu, jako je JSON.[14]

Backend (Serverová část)

Backend zajišťuje logiku aplikace, která zůstává uživateli skryta. Zahrnuje práci s databází, autentizaci uživatelů a zpracování obchodní logiky (např. vyhodnocování testů nebo ukládání výsledků). Ačkoli implementace backendu není hlavním předmětem této práce (je vyvíjen souběžně v rámci širšího projektu), je nezbytné chápat jeho roli jako poskytovatele dat, která frontend vizualizuje.[13]

Koncept Single Page Application (SPA)

Pro splnění požadavku na vysokou míru interaktivity a snížení kognitivní zátěže uživatelů byla pro vývoj zvolena architektura Single Page Application.

SPA, neboli jednostránková aplikace, je webová aplikace, která načítá pouze jeden HTML dokument. Namísto načítání celých nových stránek ze serveru při každé interakci uživatele (jak je běžné u tradičních Multi-Page Applications - MPA), SPA dynamicky přepisuje obsah aktuální stránky.[15]

Technický princip a výhody pro uživatelskou zkušenost (UX)

U tradičních webů každé kliknutí na odkaz nebo odeslání formuláře způsobí, že prohlížeč odešle požadavek na server a čeká na kompletní novou HTML stránku. To vede k „prokliknutí“ obrazovky a přerušení kontextu.[13] Naproti tomu SPA načte veškeré potřebné zdroje (HTML, CSS, skripty) při prvním spuštění nebo je dotahuje asynchronně na pozadí. Komunikace se serverem probíhá pouze na úrovni dat (typicky skrze API volání), nikoliv prezentační vrstvy.

Pro aplikaci určenou k testování studentů přináší SPA architektura zásadní výhody:

- **Plynulost a rychlost:** Přejít mezi jednotlivými otázkami v testu je okamžitý. Aplikace reaguje podobně jako nativní desktopový nebo mobilní software.
- **Eliminace rušivých elementů:** Odstranění efektu opětovného načítání stránky je klíčové pro udržení pozornosti studenta. Při testování je nežádoucí, aby technické prodlevy nebo vizuální změny narušovaly soustředění. SPA zajišťuje, že uživatelské rozhraní zůstává stabilní a mění se pouze relevantní obsah (např. znění otázky).
- **Efektivita přenosu dat:** Po počátečním načtení aplikace se sítí přenášejí pouze nezbytná data, což snižuje nároky na přenosovou kapacitu a zrychluje odezvu i na pomalejších připojeních.

3.2. Framework Flutter a multiplatformní vývoj

Pro implementaci uživatelského rozhraní jsem zvolil technologii Flutter. V kontextu moderního vývoje aplikací není Flutter klasifikován pouze jako další framework, ale je definován jako komplexní UI Toolkit vyvíjený společností Google.[16] Na rozdíl od tradičních přístupů, které se často spoléhají na propojení s nativními komponentami operačního systému, přináší Flutter vlastní vykreslovací engine a sadu komponent, což umožňuje plnou kontrolu nad každým pixelem na obrazovce.

Widgety jako základní stavební jednotka

Architektura Flutteru je postavena na striktním kompozičním modelu, kde platí pravidlo: „Všechno je Widget“. Widgety jsou základními stavebními bloky uživatelského rozhraní. Nejedná se pouze o viditelné prvky, jako jsou tlačítka, textová pole či obrázky, ale widgetem je i samotné rozvržení, zarovnání, odsazení nebo detekce gest.[16]

Tento přístup lze přirovnat ke stavebnici LEGO. Vývojář skládá komplexní uživatelské rozhraní z menších, specializovaných a znovupoužitelných komponent. Widgets ve Flutteru jsou deklarativní a neměnné – popisují, jak má UI vypadat v závislosti na aktuálním stavu aplikace. Pokud se stav změní (např. student označí odpověď v testu), framework provede „rebuild“ příslušných widgetů. Tento systém je zásadní pro dodržení konzistence designu, což je jeden z hlavních požadavků práce. Použití předpřipravených widgetů zaručuje, že ovládací prvky budou mít jednotný vzhled, chování a interakci napříč celou aplikací.

Vykreslování a absence DOM

Zásadním technickým rozdílem mezi Flutterem a tradičními webovými frameworky, jako jsou React či Angular, je způsob vykreslování. Běžné webové aplikace manipulují s objektovým modelem dokumentu (DOM) a spoléhají na prohlížeč, že vykreslí HTML a CSS elementy. Flutter tuto vrstvu obchází. Místo používání nativních systémových widgetů nebo HTML elementů si Flutter vykresluje veškerou grafiku sám.

- **Na webu:** Flutter využívá kombinaci HTML, Canvas API a WebAssembly. Aplikační logika a vykreslovací jádro jsou zkompileovány a běží nad standardními webovými API, přičemž vizuální výstup je kreslen přímo na plátno.[17]
- **Výhoda pro konzistenci:** Tím, že Flutter nepoužívá nativní prvky prohlížeče (které se mohou v každém prohlížeči mírně lišit), eliminuje problémy s nekonzistentním zobrazením. Aplikace vypadá a chová se identicky na desktopovém Chrome, mobilním Safari i v případě budoucí realizace nativní aplikaci na Androidu.[16] Pro aplikaci na správu testů to znamená, že učitelé i studenti uvidí přesně to samé rozhraní bez ohledu na to, zda k testu přistupují z počítače v učebně nebo ze svého mobilního telefonu, což minimalizuje kognitivní zátěž spojenou s adaptací na různé vizuální styly.

3.3. Programovací jazyk Dart

Dart je moderní, objektově orientovaný programovací jazyk vyvíjený společností Google, který je optimalizován pro tvorbu klientských aplikací. Jeho hlavním cílem je nabídnout produktivní prostředí pro vývoj multiplatformních aplikací s vysokým výkonem na různých cílových zařízeních, včetně webu a mobilních platforem.[18] V kontextu této práce je Dart klíčový především jako fundamentální jazyk frameworku Flutter, kterému poskytuje runtime prostředí i nástroje pro formátování, analýzu a testování kódu.

Silná typovost a bezpečnost kódu

Zásadní vlastností jazyka Dart je jeho typový systém. Jedná se o silně typovaný jazyk (type safe), který využívá statickou typovou kontrolu. To znamená, že hodnota proměnné musí vždy

odpovídat jejímu statickému typu, což umožňuje odhalit mnoho potenciálních chyb již v době kompilace, nikoliv až za běhu aplikace.[18]

Součástí moderních verzí Dartu je tzv. Sound Null Safety (bezpečnost nulových hodnot). Tento mechanismus zajišťuje, že proměnné nemohou nabývat hodnoty null, pokud to vývojář explicitně nepovolí. Díky tomu je aplikace chráněna proti jedné z nejčastějších chyb v softwarovém inženýrství – výjimkám spojeným s dereferencí nulového ukazatele (null reference exceptions). Pro vývoj aplikace pro správu testů to znamená vyšší stabilitu a spolehlivost kódu.

Asynchronní programování a uživatelská zkušenost

Komunikace s backendem prostřednictvím webového API je kritickou součástí funkčnosti aplikace. Dart pro tyto účely poskytuje robustní podporu asynchronního programování pomocí tříd Future a Stream.[18]

- **Future:** Reprezentuje výsledek asynchronní operace, která bude dokončena v budoucnu (např. stažení seznamu otázek ze serveru).
- **Stream:** Umožňuje zpracovávat posloupnost asynchronních událostí v čase.

Využití těchto konstrukcí je nezbytné pro zajištění plynulosti uživatelského rozhraní (UI). V jednovláknovém prostředí, jakým je prohlížeč, by synchronní čekání na odpověď serveru způsobilo „zamrznutí“ celé aplikace. Díky asynchronnímu modelu Dartu může aplikace během čekání na data dále reagovat a vykreslovat UI. Tato vlastnost přímo podporuje naplnění jedné z klíčových heuristik použitelnosti podle Jakoba Nielsena – Viditelnost stavu systému.[9] Místo zamrznutí aplikace lze uživateli okamžitě zobrazit indikátor načítání, čímž je student informován o tom, že systém pracuje na jeho požadavku a nedošlo k chybě. To výrazně snižuje kognitivní nejistotu a zlepšuje celkový dojem z používání aplikace.

3.4. Komunikace s backendem (REST API a JSON)

Propojení webové aplikace se světem, respektive se serverovou částí, je realizováno prostřednictvím REST API (Representational State Transfer Application Programming Interface). Jedná se o sadu pravidel a pokynů pro tvorbu webových rozhraní, která definují, jakým způsobem spolu mají různé systémy komunikovat po internetu.[14]

Volba tohoto řešení vychází z požadavku na striktní oddělení uživatelského rozhraní od datové logiky. REST není konkrétní protokol, ale architektonický styl, který využívá stávající webové standardy, primárně protokol HTTP. Díky tomuto přístupu je dosaženo tzv. volné vazby (loose coupling) mezi klientem a serverem. To v praxi znamená, že frontend a backend mohou být vyvíjeny, upravovány a škálovány nezávisle na sobě, pokud je dodrženo dohodnuté rozhraní.[19]

Principy REST a bezstavová komunikace

Klíčovým konceptem REST architektury, který aplikace využívá, je bezstavovost. To znamená, že server neuchovává žádné informace o stavu klienta mezi jednotlivými požadavky. Každý HTTP požadavek odeslaný z aplikace musí obsahovat veškeré informace nezbytné k jeho vyřízení (např. autentizační token či identifikátory testu).[14] Tento princip zjednodušuje design serverové části a zvyšuje spolehlivost aplikace, jelikož výpadek spojení nezpůsobí ztrátu „rozpracovaného“ stavu na straně serveru.

Využití HTTP metod

Komunikace je založena na standardních metodách protokolu HTTP, které definují typ prováděné operace nad zdroji (v našem případě testy, otázky, odpovědi).[19] Aplikace využívá především:

- **GET:** Tato metoda slouží k získávání dat ze serveru bez jejich modifikace. V kontextu aplikace se využívá například pro stáhnutí seznamu dostupných testů nebo načtení konkrétních otázek pro studenta.
- **POST:** Slouží k odeslání nových dat na server ke zpracování. Typickým využitím je odeslání vyplněných odpovědí studenta na konci testu nebo vytvoření nového testu učitelem.
- **PUT/PATCH:** Tyto metody jsou určeny pro aktualizaci existujících zdrojů, například při úpravě zadání otázky učitelem.

Formát dat JSON

Pro samotný přenos dat v těle HTTP požadavků a odpovědí je využit formát JSON (JavaScript Object Notation). Jedná se o textový formát, který je nezávislý na programovacím jazyce, ačkoliv vychází ze syntaxe objektů v JavaScriptu.[14]

Volba JSONu přináší pro webovou aplikaci několik zásadních výhod [19]:

- **Čitelnost:** Formát je snadno čitelný pro lidi i stroje, což usnadňuje ladění a vývoj.
- **Úspornost:** Oproti starším formátům jako XML, JSON obsahuje méně metadat, což snižuje objem přenášených dat – kritický faktor pro mobilní zařízení s omezeným datovým připojením.
- **Podpora:** JSON je standardem pro moderní webová API a framework Flutter obsahuje nativní nástroje pro jeho efektivní zpracování (serializaci a deserializaci).

3.5. Správa stavu aplikace

Pod pojmem „stav“ se v kontextu vyvíjené webové aplikace rozumí veškerá data, která určují aktuální podobu uživatelského rozhraní v daném okamžiku. V případě aplikace pro správu testů

se nejedná pouze o trvalá data uložená v databázi (např. znění otázek), ale především o dynamické proměnné existující v paměti zařízení během běhu aplikace. Typickým příkladem stavu je index aktuálně zobrazené otázky, seznam odpovědí vybraných studentem, zbývající čas časovače nebo informace o tom, zda aplikace právě načítá data ze serveru.[16]

Reaktivní model a deklarativní UI

Flutter se odlišuje od tradičních imperativních frameworků (kde vývojář manuálně manipuluje s prvky DOM) svým reaktivním a deklarativním přístupem. Základní myšlenku lze vyjádřit vzorcem:

$$\text{UI} = f(\text{state})$$

Tento vztah definuje, že uživatelské rozhraní je přímou funkcí aktuálního stavu aplikace. Vývojář nedefinuje změny UI přímo (např. „změň barvu tlačítka na modrou“), ale modifikuje stav (např. „odpověď A byla vybrána“). Framework následně automaticky detekuje tuto změnu a překreslí příslušnou část uživatelského rozhraní tak, aby odpovídala novým datům.

V praxi to znamená, že pokud student v testu klikne na odpověď:

1. Uživatelská interakce vyvolá změnu stavu (např. proměnná `selectedAnswerId` se změní z `null` na ID konkrétní odpovědi).
2. Tato mutace stavu, vyvolaná například metodou `setState()` [16], signalizuje frameworku nutnost aktualizace.
3. Widgety závislé na tomto stavu se znovu sestaví. Vizuálně se tato změna okamžitě projeví – tlačítko s vybranou odpovědí změní barvu pozadí, čímž uživatel dostává okamžitou zpětnou vazbu bez nutnosti znovunačítání stránky.

Tento mechanismus je kritický pro snížení kognitivní zátěže uživatele, neboť zajišťuje konzistenci mezi tím, co uživatel dělá, a tím, co vidí, s minimální latencí.

Architektura správy stavu a volba řešení

Zatímco pro jednoduché interakce v rámci jednoho widgetu (např. animace tlačítka) postačuje lokální správa stavu pomocí `StatefulWidget` [16], komplexní aplikace vyžaduje robustnější řešení pro sdílení dat napříč celým stromem widgetů. Předávání dat skrze konstruktory hluboko do struktury aplikace (tzv. prop drilling) by vedlo k nepřehlednému a špatně udržitelnému kódu.

Flutter nabízí několik přístupů k řešení tohoto problému, přičemž základním stavebním kamenem je `InheritedWidget` [16], který umožňuje efektivní propagaci dat k potomkům v hierarchii. Nad tímto základem vznikla řada pokročilých knihoven, jako jsou `Provider`, `Riverpod` nebo `BLoC` (Business Logic Component).

Pro účely této bakalářské práce a s ohledem na oddělení prezentační vrstvy od aplikační logiky je plánováno využití architektury založené na principu `Provider` (nebo jeho modernější iteraci `Riverpod` [20]).

Důvody pro tuto volbu:

- **Separace logiky:** Umožňuje vyčlenit logiku testu (vyhodnocování, časovač, komunikace s API) mimo samotné UI komponenty. UI se pak stará pouze o vykreslování na základě dat, která mu poskytovatel dodá.
- **Testovatelnost:** Oddělenou logiku stavu je možné snadno testovat pomocí unit testů bez závislosti na grafickém rozhraní.
- **Škálovatelnost:** Tento přístup usnadňuje budoucí rozšiřování aplikace, například přidání nových typů otázek, aniž by bylo nutné přepisovat logiku vykreslování celého testu.

4. Analýza existujících řešení

Návrh a implementace nového uživatelského rozhraní webové aplikace pro efektivní správu a tvorbu testů na středních školách vyžaduje podrobnou analýzu stávajících platform. Vzdělávací technologie se neustále vyvíjejí, ale zpětná vazba od pedagogů a studentů ukazuje, že řada řešení naráží na problém, jak optimálně skloubit pedagogickou efektivitu s kvalitním uživatelským prostředím (UX/UI designem).

Hlavním měřítkem, kterým jsou analyzované systémy hodnoceny, je schopnost minimalizovat vnější kognitivní zátěž a udržet vysokou validitu didaktického testu. Namísto opětovného definování těchto jevů (podrobně rozebráno v podkapitole 1.5) se analýza přímo zaměřuje na to, jak jednotlivé platformy v praxi implementují Nielsenovy heuristiky použitelnosti, jak podporují koncept testovacího efektu prostřednictvím formativního opakování a nakolik pedagogům usnadňují flexibilní tvorbu obsahu.

Pro účely komplexního zhodnocení byl trh rozdělen do tří hlavních kategorií: globální systémy (LMS), lokální školní informační systémy (ŠIS) a komerční vzdělávací platformy (SaaS).

4.1. Globální systémy pro řízení výuky (LMS)

Moodle

Moodle představuje celosvětově nejrozšířenější open-source platformu typu LMS (Learning Management System). Z pedagogického hlediska nabízí bezkonkurenční hloubku nástrojů pro tvorbu testů, což je žádoucí zejména pro přírodovědné obory. Zásadním problémem platformy je však její komplikované uživatelské rozhraní. Tvorba tzv. vypočítávaných úloh sice umožňuje vygenerovat unikátní test pro každého studenta, ale časová a mentální náročnost na straně učitele je enormní. Systém vyžaduje zadávání vzorců ve specifické syntaxi, kde jakýkoliv překlep vede k nefunkčnosti úlohy. Rozhraní pro editaci otázek je navíc přehlcené volbami, což přímo porušuje Hickův zákon (viz kap. 2.3.1) a učitele odrazuje od častějšího formativního testování.

Pro studenty představuje Moodle vizuálně těžkopádné prostředí postrádající jasnou hierarchii. Jelikož systém tradičně využívá architekturu Multi-Page Application (MPA), každé uložení odpovědi vyžaduje opětovné načtení HTML stránky. To, jak bylo rozebráno v kapitole 3.1, narušuje plynulost, způsobuje rušivé problikávání a zvyšuje stres z ubíhajícího času.

Google Classroom

Na opačném konci spektra stojí Google Classroom, navržený s důrazem na absolutní jednoduchost a čistý Material Design. Zásadní slabinou je však skutečnost, že se platforma pro testování plně spoléhá na externí nástroj Google Forms. Typy úloh jsou omezeny na základní varianty (nejčastěji používané Multi Choice Question, MCQ) a systém postrádá hlubší pedagogickou analytiku či automatizovanou podporu pro adaptivní opakování.

Microsoft Teams for Education

Podobně jako Google, i Teams využívá pro testování externí aplikaci (Microsoft Forms). Výhodou je silné propojení s ekosystémem Office 365 – studenti testy otevírají přímo uvnitř aplikace, čímž se snižuje kognitivní zátěž spojená s přepínáním oken. Z pohledu UX je ale nejčastějším problémem informační přehlčení. Neustálé konverzace a notifikace v komunikační platformě vytvářejí dynamické prostředí, které snadno naruší soustředění potřebné pro řešení testu. Administrace virtuálních tříd je navíc poměrně nepružná a časově náročná.

Tabulka 4.1.: Srovnání systémů pro správu testů z hlediska UX/UI

Parametr hodnocení UX/UI	Moodle	Google Classroom	MS Teams for Education
Primární zaměření	Komplexní e-learning	Rychlá distribuce úkolů	Synchronní komunikace
Testovací architektura	Interní, hluboce provázaná	Externí (Google Forms)	Externí (Microsoft Forms)
Kognitivní zátěž učitele	Vysoká	Velmi nízká	Střední (složitě řízení skupin)
Typologie úloh	Široká	Omezená (MCQ, text)	Omezená (MCQ, text)

4.2. Lokální školní informační systémy (ŠIS)

Velká část elektronického testování v ČR probíhá v lokálních systémech, které historicky vznikly primárně jako nástroje pro administrativu (třídní knihy, rozvrhy).

Bakaláři a Škola Online

V systému Bakaláři vizuální a funkční podoba e-learningu stále silně odráží jeho původní tabulkovou povahu. Vytvořit vizuálně bohatý test pro přírodovědné obory je velmi obtížné. Testovací prostředí pro studenty působí staticky a chybí mu plynulá okamžitá zpětná vazba, což omezuje

jeho využití pro formativní hodnocení. Z hlediska webové architektury vyžaduje orientace v systému nadbytečné množství interakcí a opakované načítání celých dokumentů ze serveru, což prokazatelně narušuje plynulost práce. Podobně je na tom Škola Online, jejíž e-learningový modul je navržen spíše z pohledu administrátora než s ohledem na studenta.

EduPage

Slovenský systém EduPage je architektonicky i funkčně mnohem modernější. Nabízí interaktivní karty a nativní podporu pro široké spektrum úloh včetně složitých zlomků. Velkou výhodou je automatizované generování variant testů, které brání opisování bez nutnosti složitého nastavování. Uživatelská zkušenost na mobilních zařízeních je plynulá a aplikace obsahuje i účinný systém ochrany proti podvádění.

4.3. Komerční vzdělávací platformy (SaaS)

Třetí významnou kategorii představují nezávislé webové platformy fungující v modelu software jako služba (Software as a Service, SaaS). Tyto systémy, mezi něž patří například Kahoot!, Socrative nebo Quizlet, jsou navrženy s primárním cílem maximalizovat zapojení žáků a nabídnout bezbariérovou uživatelskou zkušenost bez nutnosti složité integrace do školní infrastruktury.

Kahoot! a Quizlet

Kahoot! představuje etalon v oblasti gamifikace výuky. Z pohledu UX na mobilních zařízeních je systém navržen excelentně – plně respektuje Fittsův zákon (obrovská barevná tlačítka pro volbu odpovědi) a minimalizuje vnější kognitivní zátěž spojenou s ovládáním na naprosté minimum.

Z hlediska validity středoškolského didaktického testu však naráží na vážné limity. Striktní časový odpočet a silně soutěživý charakter aplikace výrazně zvyšují stres žáků, což vede k impulzivnímu hádání namísto analytického řešení problému. Aplikace postrádá prostor pro poskytnutí kvalitní procesuální zpětné vazby (viz kap. 1.3), čímž je narušen formativní testovací efekt. Tvorba úloh pro přírodovědné předměty je navíc ztížena velmi omezenou podporou složitých vzorců a formátování.

Socrative

O něco pedagogicky využitelnější variantu představuje platforma Socrative. Z uživatelského hlediska využívá moderní responzivní rozhraní, které eliminuje vizuální balast. Podporuje testy řízené vlastním tempem studenta, což žákům poskytuje klid pro soustředění a umožňuje lepší uplatnění formativního hodnocení včetně otevřených otázek.

Zásadním limitem této i dalších podobných SaaS platform pro běžnou školní praxi je ovšem jejich izolovanost. Učitelé musí databázi žáků a exporty výsledků spravovat odděleně od hlavního

školního informačního systému (Bakaláři/EduPage), což radikálně zvyšuje jejich administrativní zátěž.

4.4. Architektonická a technologická východiska pro návrh nového frontendu

Zastaralý vzhled a pomalejší interakce tradičních školních systémů jsou dány primárně jejich MPA architekturou. Z psychologického hlediska toto neustálé technické přerušování narušuje hluboké soustředění žáka. Z analýzy proto vyplývají následující požadavky na vývoj nového systému, které úzce korespondují s technologiemi vybranými v kapitole 3:

- **Implementace architektury SPA:** Jak bylo teoreticky zdůvodněno, nasazení Single Page Application eliminuje technologickou nejistotu a rušivé elementy spojené s opětovným načítáním stránek. Okamžitá reaktivita přímo naplňuje Nielsenovu heuristiku viditelnosti stavu systému.
- **Řešení vizuální fragmentace pomocí Flutteru:** Mobilní verze stávajících platforem často zobrazují obsah chybně. Flutter, využívající vlastní vykreslovací engine, garantuje vizuální identitu složitých vzorců napříč platformami. Dodržení principu mobile-first navíc v kombinaci s Fittsovým zákonem řeší problém nepřesných dotyků.
- **Okamžitá interakce skrze notifikace:** Namísto nutnosti manuálně a opakovaně otevírat aplikaci za účelem zjištění novinek (např. neustálým obnovováním stránky) bude systém uživatele aktivně upozorňovat prostřednictvím push notifikací v reálném čase. Tím se zásadně zlepši informovanost studentů o nově zadáných testech či výsledcích.
- **Multimediální podpora:** Aplikace musí pedagogům nabídnout intuitivní editory pro snadné vkládání obrázků a schémat, čímž se radikálně sníží časová náročnost přípravy.

5. Analýza požadavků a návrh uživatelského rozhraní

Návrh a implementace webové aplikace pro efektivní správu a tvorbu testů vyžaduje detailní analýzu potřeb všech zainteresovaných stran. V kontextu této bakalářské práce jsou primárními zadavateli pedagogové – konkrétně učitelé přírodovědných předmětů. Sekundární, avšak z hlediska uživatelské zkušenosti neméně kritickou cílovou skupinou, jsou samotní žáci. Na základě analýzy stávajících platforem (viz kap. 4) a s ohledem na nutnost minimalizace kognitivní zátěže byly definovány požadavky, které následně posloužily k vytvoření interaktivních prototypů. Pro vizualizaci a definici interakčních toků byl využit designový nástroj FlutterFlow, jehož volba přímo koresponduje s plánovanou technologickou implementací ve frameworku Flutter.

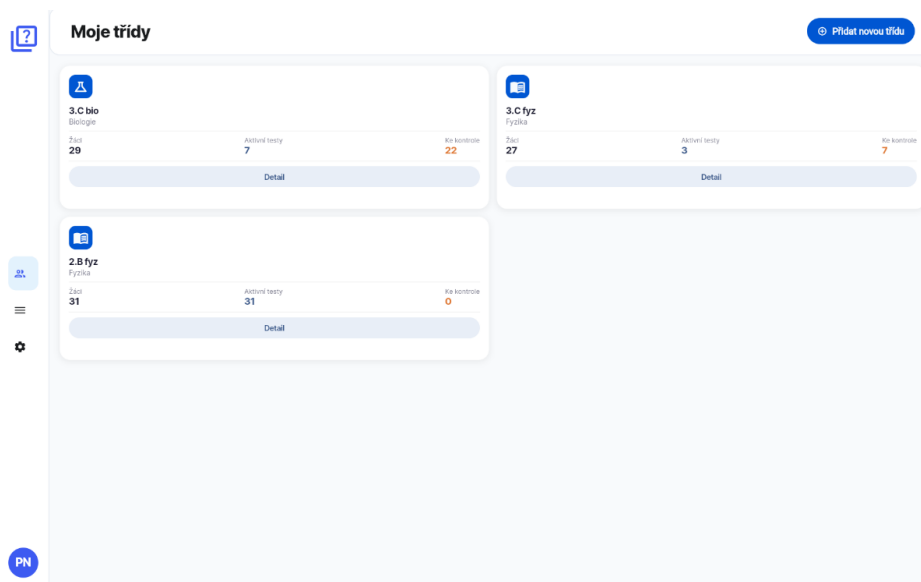
5.1. Identifikace cílových skupin a oddělená rozhraní

Pochopení mentálních modelů obou stran vzdělávacího procesu je výchozím bodem pro návrh aplikace. Základním architektonickým požadavkem je striktní oddělení uživatelských rozhraní pro učitele a pro žáky. Sloučení obou pohledů do jednoho celku by vedlo k nárůstu vizuálního smogu a porušení heuristiky [9] minimalistického designu. Vstupním bodem do aplikace je tak přehledná přihlašovací obrazovka, kde si uživatel pomocí zřetelného segmentového přepínače volí svou roli. Toto řešení funguje jako prevence chyb – okamžitě filtruje dostupné funkce a směřuje uživatele do příslušného, izolovaného pracovního prostředí.

5.2. Rozhraní pedagoga: Správa testů a tvorba obsahu

Učitelské rozhraní musí plnit roli komplexního řídicího centra, které pedagogům šetří čas a minimalizuje administrativu.

- **Řídicí panel:** Po přihlášení se učiteli zobrazí panel „Moje třídy“, který k prezentaci dat využívá modulární systém karet (viz Obr. 5.1). Architektura logicky shlukuje funkce do tematických celků podle Hickova zákona [10] (viz kap. 2.3.1). Každá karta nabízí rychlý přehled metrik (počet žáků, aktivní testy) a barevná signalizace intuitivně navádí pozornost k prioritním úkolům. Detailní analytika a přehled testů je následně k dispozici po rozkliknutí konkrétní třídy (viz Obr. 5.2).



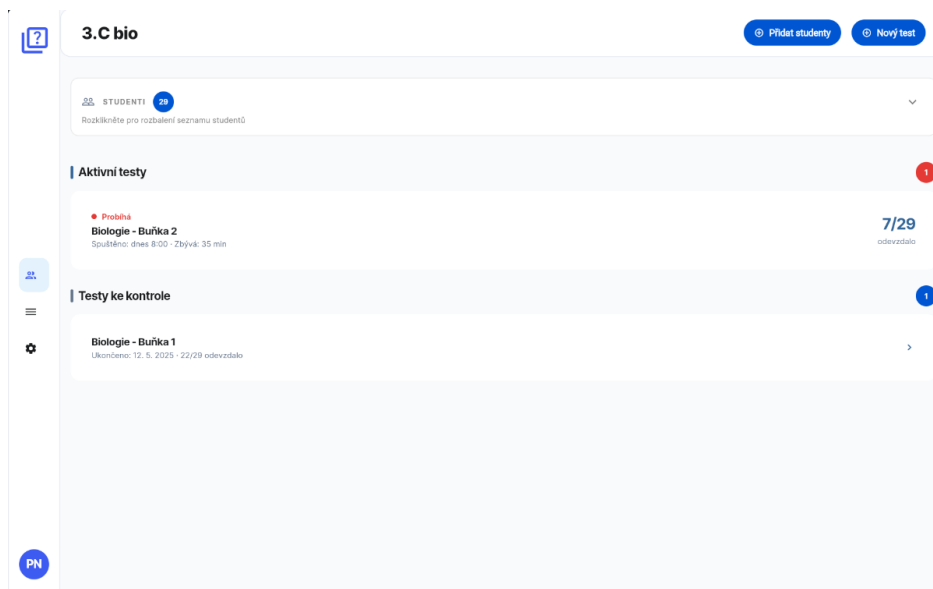
Obrázek 5.1.: Návrh hlavního řídicího panelu učitele s využitím modulárního systému karet pro minimalizaci kognitivní zátěže.

- **Banky otázek a multimédia:** V kontextu přírodovědných předmětů je nepostradatelným požadavkem bezproblémové vkládání obrázků a schémat. Aplikace poskytuje intuitivní drag-and-drop editor (viz Obr. 5.3). Formulář se dynamicky přizpůsobuje zvolenému typu otázky. Klíčovým prvkem je tlačítko „Pohled studenta“, které učiteli umožňuje kdykoliv ověřit vizuální podobu úlohy z perspektivy žáka.
- **Sestavení testu a parametry učení:** Samotný test učitel sestavuje výběrem z bank otázek (viz Obr. 5.4). Rozhraní obsahuje srozumitelné přepínače pro definici okna přístupnosti a časového limitu. Možnost nastavit počet pokusů transformuje aplikaci ze sumativního měřicího nástroje na formativní trénink, který aktivně podporuje testovací efekt [3] (viz kap. 1.2).

5.3. Rozhraní žáka: Testovací proces a mobilní optimalizace

Zatímco učitelské rozhraní snese vyšší informační hustotu, prostředí pro žáky klade absolutní důraz na minimalismus a princip mobile-first design (viz kap. 1.7).

- **Rozcestník žáka:** Po přihlášení studenta se zobrazí čistý a minimalistický hlavní panel. Pozornost je okamžitě směřována k samotnému vrchu obrazovky na sekci „Aktivní testy“ (viz Obr. 5.5), kde jsou úkoly s blížícím se termínem vizuálně zvýrazněny. Zbytek hlavní stránky tvoří přehledový seznam jednotlivých tříd. Po rozkliknutí konkrétní třídy se žák dostane k historii již splněných testů a svým dosaženým výsledkům (viz Obr. 5.6). Tento zjednodušený, plochý design udržuje přímý tok informací a zcela odstraňuje nutnost zdlouhavého hledání v nepřehledných strukturách složek.
- **Izolovaný testovací režim a přístupnost:** Při spuštění testu aplikace skryje globální navigaci i postranní panely (viz Obr. 5.7). Na obrazovce zůstává výhradně znění otázky,



Obrázek 5.2.: Detailní analytika konkrétní třídy s barevnou signalizací prioritních úkolů a přehledem aktivních testů.

odpovědi a indikátor průběhu. Pro zajištění inkluzivního prostředí aplikace zaručuje vysoký kontrast, škálovatelnost písma a možnost uživatelské aktivace tmavého režimu pro snížení zrakové únavy.

- **Ergonomie a typologie úloh:** Odpovědní pole u úloh typu multiple-choice jsou realizována jako široké dotykové karty s doporučenou minimální velikostí 48x48 dp (tzv. *Density-independent Pixels*). Tato hodnota vychází z oficiálních doporučení Material Designu pro minimální velikost dotykových ploch. Tím je plně respektován Fittsův zákon [11] (viz kap. 2.3.2) a eliminuje se problém nepřesných dotyků (fat-finger problem). U přiřazovacích úloh systém nabízí responzivní roletová menu namísto složitého přesouvání prvků na malém displeji.
- **Časový management a stabilita:** Vizuální časovač v periferní části obrazovky se barví varovně červeně až v posledních minutách, aby se minimalizovala počítačová úzkost (viz kap. 1.4). Přejít mezi otázkami probíhá asynchronně a okamžitě (díky architektuře SPA). Systém navíc využívá ukládání dat do lokální mezipaměti jako toleranci vůči výpadkům školní sítě.
- **Prevence chyb při odevzdávání:** Před finálním odesláním testu aplikace vyvolá potvrzovací dialog, čímž chrání žáka před nechtěným přehmatem a předčasným odevzdáním.

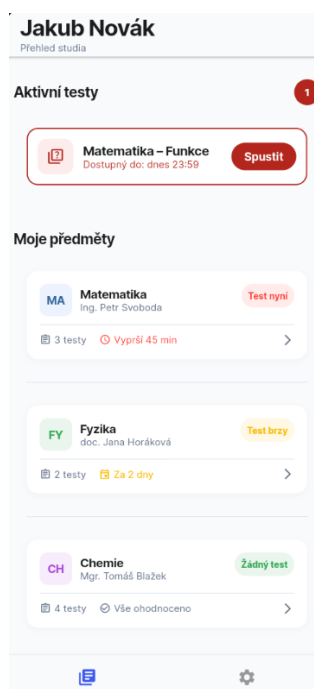
5.4. Notifikace a formativní zpětná vazba

Zásadní inovací je proaktivní komunikace prostřednictvím push notifikací v reálném čase. Žák již nemusí systém pasivně kontrolovat; aplikace ho sama upozorní na nový test nebo opravené úlohy. Aby se předešlo únavě z oznámení, uživatelé si mohou sami definovat, jaký typ událostí chtějí odebírat.

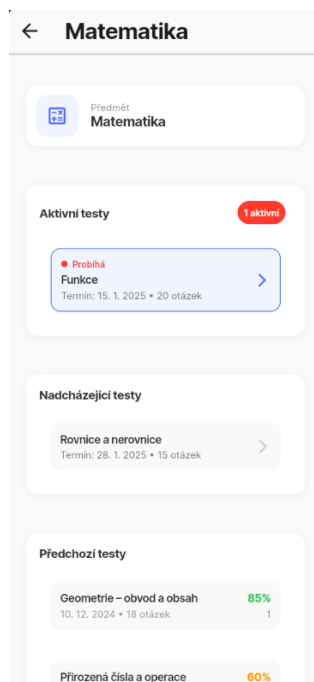
Obrázek 5.3.: Intuitivní editor testových úloh s dominantní drag-and-drop zónou pro multimédia a funkcí náhledu z perspektivy žáka.

- **Okamžitá zpětná vazba:** Pokud to učitel v nastavení povolí, žák obdrží bezprostředně po označení odpovědi asynchronně vyhodnocenou volbu s textovým vysvětlením. Tento princip okamžitosti přímo naplňuje Hattieho model zpětné vazby [4] (viz kap. 1.3) a zabraňuje fixaci chybných úvah.
- **Hodnocení otevřených úloh:** Pro kontrolu otevřených úloh má učitel k dispozici textové pole pro zápis slovního formativního hodnocení (viz Obr. 5.8). Jakmile zapíše body a hodnocení, výsledek se okamžitě synchronizuje do databáze a žákovi je odeslána notifikace.

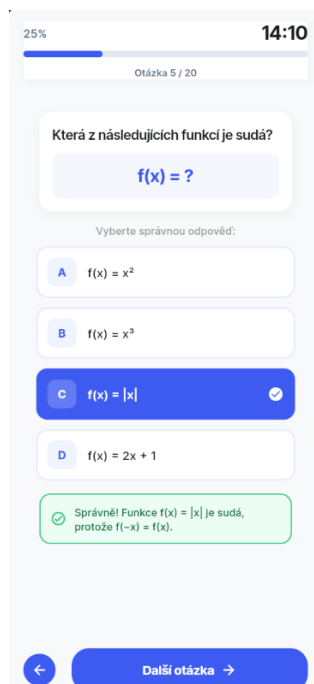
Obrázek 5.4.: Konfigurační rozhraní pro sestavení testu s přepínači definujícími parametry didaktického cyklu a formativní zpětné vazby.



Obrázek 5.5.: Minimalistický rozcestník žáka. Pozornost je primárně směřována na sekci aktivních testů s blížícím se termínem odevzdání.



Obrázek 5.6.: Detail konkrétního předmětu poskytující žákovi okamžitý přehled o historii splněných testů a dosažených výsledcích.



Obrázek 5.7.: Izolovaný testovací režim. Rozhraní eliminuje vizuální smog a využívá široké dotykové karty pro minimalizaci problému nepřesných dotyků na mobilním zařízení.

The screenshot displays a user interface for a teacher's control panel. At the top, the user is identified as 'Jan Zápotocký' (Biologie - Buňka 1) and the date is '12.05.2025'. The interface is divided into two main sections. The first section, '1. Výběr z možností', contains a multiple-choice question: 'Co je energetickým centrem buňky?'. The options are A. Neutrony, B. Mitochondrie (selected and marked correct with a green check), C. Plazmidy, and D. Neuroity. The second section, '2. Otevřená otázka', contains an open question: 'Stručně popište funkci buněčné membrány.' Below the question, there is a text input area with a pre-filled answer: 'Buněčná membrána funguje jako bariéra, která kontroluje, co vstupuje do buňky a co ji opouští. Je to jako vrátný. Udrží vnitřek buňky oddělený od vnějšího prostředí.' To the right of the input area, there is a 'Udělené body' (Assigned points) field showing the value '3'. At the bottom right, there is a blue button labeled 'Uložit hodnocení' (Save evaluation). On the left side of the interface, there is a vertical sidebar with icons for help, a list of questions, and a settings gear.

Obrázek 5.8.: Rozhraní pedagoga pro kontrolu testů. Návrh kombinuje automaticky vyhodnocené úlohy s prostorem pro manuální zápis bodů a individuální formativní zpětné vazby k otevřeným otázkám.

5.5. Shrnutí funkčních a nefunkčních požadavků

Předcházející analýza potřeb a na ni navazující návrh uživatelského rozhraní definují závazný rámec pro softwarovou implementaci aplikace.

Tabulka 5.1.: Matice vybraných funkčních požadavků

Kategorie funkce	Specifikace požadavku	Přínos pro uživatelskou zkušenost
Architektura uživatelů	Dvě striktně oddělená a nezávislá přístupová rozhraní.	Ochrana před vizuálním a kognitivním přetížením.
Multimediální prvky	Modul pro vkládání komprimovaných obrázků do zadání i odpovědí.	Zásadní požadavek pro vizualizaci ve výuce přírodních věd.
Didaktické cykly	Možnost definovat povolený počet pokusů na zpracování testu.	Transformace zkoušení na formativní nástroj výuky (Testing Effect [3]).
Doručování zpráv	Distribuce kontextových push notifikací v reálném čase.	Aktivní řízení času žáka namísto pasivní kontroly systému.

Tabulka 5.2.: Matice vybraných nefunkčních požadavků

Kategorie vlastností	Specifikace požadavku	Očekávaný technologický či interakční dopad
Plynulost a latence	Eliminace nutnosti znovunačítání stránky při průchodu testem.	Implementace moderní Single Page Application (SPA).
Responzivní přístupnost	Plná ovladatelnost na malých dotykových displejích.	Aplikace strategie mobile-first a Fittsova zákona.
Tolerance vůči výpadkům	Aplikace neztratí data při krátkodobém výpadku připojení.	Lokální cachování do mezipaměti před synchronizací.
Vizuální minimalismus	V testovacím režimu rozhraní skrývá irelevantní menu a hlavičky.	Izolace řešené úlohy a přísné snížení vnější kognitivní zátěže.

6. Proces návrhu a využití UX/UI nástrojů

Předcházející analýza cílových skupin a specifikace funkčních i nefunkčních požadavků (kap. 5) definovala mantinely, které musí nově vznikající systém bezpodmínečně splňovat. Aby však bylo možné tyto teoretické premisy úspěšně přetavit do podoby funkčního zdrojového kódu, je nezbytné aplikovat strukturovaný proces designu. Přímý přechod od specifikace požadavků k samotnému programování frontendu s sebou nese vysoké riziko vzniku nepřehledného rozhraní, které by nerespektovalo kognitivní limity svých uživatelů.

Z tohoto důvodu byla mezi fází analýzy a fází samotné implementace (kap. 7) vložena kritická etapa návrhu uživatelské zkušenosti (UX) a tvorby vizuálního rozhraní (UI). Právě tato etapa se zaměřuje na to, jakým způsobem byly teoretické poznatky z oblasti kognitivní psychologie a interakčního designu aplikovány v praxi. Popisuje výběr vhodných návrhových nástrojů, metodu rychlého prototypování (Rapid Prototyping) s přechodem k High-Fidelity modelům a definici jednotného design systému. Cílem tohoto postupu bylo vytvořit sadu vizuálních a interakčních pravidel, která garantují, že výsledná aplikace bude pro pedagogy maximálně efektivní a pro studenty intuitivní.

6.1. Definice cílových uživatelů

Jakýkoliv úspěšný návrh uživatelského rozhraní musí vycházet z hlubokého pochopení koncových uživatelů. Pro zajištění toho, aby aplikace skutečně řešila reálné problémy a nezvyšovala kognitivní zátěž zúčastněných stran, byla využita metoda tvorby uživatelských person (User Personas). Persony představují archetypy reálných uživatelů a slouží jako primární referenční bod při rozhodování o vizuální hierarchii, interakcích a prioritizaci funkcí. Vzhledem k zaměření tohoto projektu na gymnázia byly definovány dva klíčové mentální modely.

6.1.1. Persona 1: Učitel přírodovědných předmětů

Primárním zadavatelem a správcem obsahu v aplikaci je pedagog. Z analýzy požadavků vyplynulo, že učitelé jsou často přetíženi administrativou a stávající e-learningové platformy u nich vyvolávají frustraci kvůli nadměrné složitosti.

- **Hlavní potřeby:** Učitel potřebuje především efektivitu a rychlost. Vyžaduje nástroj, který mu umožní vytvořit test během několika minut, aniž by musel procházet hluboce vnořenými

menu. Vzhledem k přírodovědnému zaměření je pro něj kritická možnost intuitivně vkládat do testů grafy a schémata.

- **Frustrace:** Odmítá složité proklikávání, redundantní potvrzování akcí a nepřehlednou analytiku.
- **Dopad na design:** Rozhraní pro učitele je koncipováno jako komplexní, ale přehledný řídicí panel (dashboard). Využívá modulární karty, které na první pohled poskytují analytická data o konkrétní třídě. Tvorba otázek je řešena pomocí dynamických formulářů, čímž se radikálně zkracuje čas potřebný k přípravě učiva.

6.1.2. Persona 2: Středoškolský student

Zatímco učitel je tvůrcem obsahu, student je jeho hlavním konzumentem. Mentální model středoškolského studenta je z pohledu testování často spojen se stresem a časovým tlakem.

- **Hlavní potřeby:** Student vyžaduje absolutní srozumitelnost a přímý tah na cíl. Potřebuje přesně vědět, jaké testy má aktivní, do kdy je musí odevzdat a kolik času mu zbývá během samotného řešení. Dalším klíčovým aspektem je preference mobilních zařízení – očekává, že test dokáže plnohodnotně a bez chyb vyplnit na displeji svého chytrého telefonu.
- **Frustrace:** Ztráta koncentrace způsobená vizuálním balastem, nepřehledná navigace nebo strach z nechtěného (předčasného) odevzdání testu v důsledku špatného kliknutí.
- **Dopad na design:** Žákovské rozhraní striktně aplikuje principy mobile-first designu a minimalismu. Při spuštění testu aplikace přechází do izolovaného režimu, který skrývá veškerou okolní navigaci a snižuje vnější kognitivní zátěž. Součástí rozhraní jsou také jasně viditelné časovače a záchranné mechanismy v podobě potvrzovacích dialogů, které studenta chrání před nevratnými chybami.

6.2. Nástroje pro návrh a prototypování

Základním předpokladem pro vytvoření kvalitní uživatelské zkušenosti je iterativní proces návrhu. Ten umožňuje testovat, upravovat a ověřovat designová rozhodnutí dříve, než dojde k samotné softwarové implementaci, čímž se radikálně snižují náklady na případné úpravy zdrojového kódu. Pro tento účel bylo nezbytné zvolit odpovídající návrhové UX nástroje a definovat ucelenou informační architekturu.

Volba prototypovacího nástroje: FlutterFlow

Pro tvorbu interaktivních prototypů byl na rozdíl od tradičních grafických editorů (např. Figma) zvolen nástroj FlutterFlow [21]. Hlavním argumentem pro tuto volbu byla technologická synergie.

Jelikož cílová aplikace využívá pro svůj běh framework Flutter (viz kap. 3.2), umožňuje FlutterFlow navrhovat uživatelské rozhraní přímo s využitím nativních Flutter widgetů.

Tento přístup přinesl dvě zásadní výhody. Za prvé, navržený design nebyl pouhou statickou grafikou, ale interaktivním modelem s reálnou fyzikou Material Designu (odezvy tlačítek, animace posunů). Za druhé, designová omezení nebo naopak možnosti nástroje FlutterFlow přesně reflektovaly technické možnosti výsledné aplikace. Tím byla zajištěna stoprocentní proveditelnost navrženého UX do finálního zdrojového kódu.

Odklon od drátěných modelů k Rapid Prototypingu (High-Fidelity)

Tradiční proces návrhu uživatelského rozhraní často začíná tvorbou tzv. drátěných modelů (wireframes) – abstraktních schémat s nízkou věrností zobrazení (Low-Fidelity). V rámci tohoto projektu byl však tento krok záměrně přeskočen a byl zvolen přístup tzv. rychlého prototypování (Rapid Prototyping) s přímým přechodem do návrhů s vysokou věrností (High-Fidelity).

Tento postup byl umožněn právě díky zvolenému nástroji FlutterFlow, který disponuje robustní knihovnou hotových vizuálních komponent vycházejících z Material Designu. Místo abstraktních obdélníků tak byla rovnou sestavována logická struktura (informační architektura) tak, aby odpovídala definovaným mentálním modelům person (viz kap. 6.1), a to rovnou za pomoci reálných vizuálních prvků, barev a typografie. Výhodou tohoto přístupu bylo, že navrhované rozhraní od prvního momentu poskytovalo přesnou vizuální i funkční zpětnou vazbu.

I při tomto zrychleném postupu však byla striktně dodržována promyšlená informační architektura. Klíčová architektonická rozhodnutí učiněná během prototypování zahrnují:

1. **Modulární systém karet (Dashboard učitele):** Informační architektura řídicího panelu učitele byla navržena tak, aby agregovala data. Místo dlouhých seznamů a tabulek byly využity prostorové moduly (karty) reprezentující jednotlivé třídy. Učitel tak nemusí hledat informace v hlubokých strukturách menu, ale vidí okamžitý přehled počtu žáků a aktivních testů přímo na uvítací obrazovce.
2. **Izolovaný testovací režim (Prostředí žáka):** V případě žakovského rozhraní byla architektura podřízena požadavku na minimalizaci rozptylování. Modely pro samotný testovací proces byly záměrně zbaveny všech globálních navigačních prvků (hlavičky, postranní panely). Vznikl tak „izolovaný režim“, kdy obrazovka obsahuje výhradně znění otázky, odpovědní zóny a stavové indikátory (časovač a průběh testu). Tímto rozhodnutím byla aplikována Nielsenova heuristika pro minimalismus [9] a maximalizována vizuální pozornost žáka.

6.3. Tvorba design systému (Design System & UI Kits)

Kvalitní uživatelská zkušenost nevzniká náhodným vrstvením grafických prvků, ale vyžaduje striktní systémovost. Aby byla zajištěna absolutní vizuální konzistence napříč celou webovou

aplikací (od přihlašovací obrazovky až po vyhodnocení testu), byl na počátku procesu definován vlastní design systém. Tento ucelený soubor vizuálních pravidel funguje jako stavebnice (UI Kit) definující vzhled tlačítek, karet, barev i typografie.

Aplikace principů Material Design 3

Jako fundamentální vizuální jazyk byl zvolen Material Design 3 (MD3) [12] od společnosti Google. Tento designový rámec byl využit především pro svou schopnost imitovat fyzikální vlastnosti reálného světa v digitálním prostředí. Zásadním prvkem aplikovaným v návrhu je vizuální hierarchie pomocí stínování. Karty reprezentující jednotlivé třídy nebo bloky s otázkami jsou graficky „vyvýšeny“ nad pozadí pomocí jemných stínů, čímž vzniká pocit hmatatelnosti a prostorového oddělení. V kombinaci se zakulacenými rohy (border radius) působí rozhraní méně stroze a přispívá k pocitu uživatelské přívětivosti.

Barevná paleta a kódování informací

Barevnost aplikace není navržena pouze z estetického, ale primárně z funkčního hlediska (psychologie barev). Základní paleta rozhraní staví na neutrálních světlých odstínech pozadí s výraznou primární (modrou) barvou pro hlavní akční tlačítka, což pomáhá vést zrak uživatele.

Zcela stěžejní roli však hrají barvy při kategorizaci obsahu, konkrétně u štítků označujících typy testových otázek. Aby systém učiteli i žákovi maximálně usnadnil vizuální orientaci při procházení testu, byla pro štítky vytvořena paleta jemných pastelových barev s kontrastním probarvením textu:

- **Modrá:** Úlohy s výběrem z možností (Single/Multi choice).
- **Oranžová:** Uspořádací a seřazovací úlohy.
- **Zelená:** Otevřené textové úlohy.
- **Červená:** Úlohy s krátkou odpovědí.
- **Fialová:** Párovací a spojovací úlohy.

Volba pastelových tónů namísto vysoce sytých (křiklavých) barev není náhodná. Agresivní barvy by při větším množství otázek na jedné obrazovce způsobovaly vizuální únavu a odváděly pozornost od samotného znění úlohy. Navržené tlumené kódování naopak působí uklidňujícím dojmem, nesoutěží s hlavním obsahem, a přesto poskytuje dostatečný kontrast pro rychlou orientaci (tzv. skenovatelnost textu).

Typografie a čitelnost

Vzhledem k tomu, že primárním obsahem aplikace jsou dlouhé texty (zadání testových úloh, možnosti odpovědí a studijní texty), je typografie kritickým faktorem použitelnosti. Pro celý systém byla zvolena rodina bezpatkových (sans-serif) písem Inter.

Tento moderní font byl navržen speciálně pro počítačové obrazovky. Vyznačuje se vynikající čitelností i v menších velikostech a eliminuje zbytečné dekorativní prvky, které by na displejích mobilních telefonů s nižším rozlišením mohly splývat. Tímto rozhodnutím byla opět minimalizována kognitivní zátěž uživatele, neboť zrak nemusí vynakládat dodatečné úsilí na rozluštění tvaru jednotlivých písmen.

6.4. Mapování uživatelských cest a interakce

Vizuální design samotných obrazovek představuje pouze statickou část uživatelské zkušenosti. Neméně důležité je to, jak plynule uživatel těmito obrazovkami prochází (tzv. User Flow) a jak s ním systém komunikuje. Špatně navržená posloupnost kroků může vést k dezorientaci, frustraci a tzv. kognitivnímu ztracení (uživatel neví, kde se nachází nebo jak se vrátit zpět). Aby se tomuto jevu předešlo, byly pro obě hlavní cílové skupiny detailně zmapovány a optimalizovány jejich interakční toky.

Tok učitele

Základním procesem v učitelském rozhraní je příprava a zadání testu. Cílem bylo tento tok maximálně zkrátit a odstranit zbytečné mezikroky, které brzdí plynulost práce. Učitel po úspěšném přihlášení vstupuje na primární řídicí panel (Dashboard). Odtud vede jeho cesta přímým výběrem konkrétní třídy.

Místo přesměrování do složitých administračních formulářů se mu otevírá intuitivní editor testů. Samotná editace a kompletace otázek z bank probíhá asynchronně – systém nevyžaduje znovunačítání celé stránky při přidání každé nové položky, čímž uživatel neztrácí kontext rozpracovaného úkolu. Finální akce (uložení a přiřazení testu studentům) probíhá na pozadí, přičemž je učitel ihned informován prostřednictvím vizuální zpětné vazby v podobě „toast“ notifikace (zelená bublina s potvrzením o úspěšném zadání). Celý proces je tak lineární a nevyžaduje opuštění primárního interakčního toku.

Tok studenta

Interakční trasa žáka má zcela odlišnou dynamiku a klade absolutní důraz na soustředění a prevenci chyb. Flow studenta začíná na rozcestníku, kde je mu vizuálně zvýrazněn test vyžadující pozornost (např. test s blížícím se termínem).

Po kliknutí na spuštění testu dochází k zásadnímu interakčnímu posunu – student přechází do tzv. „izolovaného testovacího režimu“. Systém záměrně uzamkne boční navigaci i horní lišty a uvězní uživatele v prostředí, kde existuje pouze zadání úlohy, možnosti odpovědí a jasný časový odpočet. Kognitivní kapacita žáka je tak plně vyhrazena na luštění úlohy, nikoliv na orientaci v menu.

Kritickým bodem celého toku je fáze odevzdání. Aby se předešlo fatálním omylům (Nielsenova heuristika prevence chyb [9]), je implementována ochrana proti předčasnému nechtěnému odevzdání. Jakýkoliv pokus o finální uložení (nebo opuštění obrazovky) vyvolá modální potvrzovací dialog, který žáka upozorní na nezodpovězené otázky. Teprve po vědomém potvrzení je test ukončen a tok okamžitě navazuje zobrazením výsledkové obrazovky s formativní zpětnou vazbou (v případě, že tak učitel test nastaví). Z testování se tak stává plynulá křivka: soustředění → potvrzení → okamžité poučení z chyb.

6.5. Optimalizace pro použitelnost

V samotném závěru designového procesu bylo nezbytné podrobit navržené rozhraní kritické revizi a aplikovat konkrétní optimalizační kroky. Teoretická východiska popsaná v kapitole 2, definující limity lidské kognice a motoriky, nezůstala v rovině abstraktních doporučení, nýbrž se přímo propsala do finální podoby jednotlivých UI prvků. Cílem bylo zajistit absolutní plynulost používání a vyhnout se jevům, které by snižovaly validitu testovacího procesu. Dva stěžejní psychologické principy, které do formování uživatelského rozhraní zasáhly nejvýrazněji, jsou Hickův a Fittsův zákon.

Praktická aplikace Hickova zákona [10]

Hickův zákon stanovuje, že čas potřebný k rozhodnutí exponenciálně roste s počtem nabízených možností. Čím složitější je nabídka, tím vyšší je kognitivní zátěž uživatele. Tento princip byl do aplikace promítnut formou radikálního minimalismu, a to zejména v rozhraní pro studenty.

Tradiční e-learningové platformy mají tendenci na úvodní obrazovce zobrazovat desítky odkazů, nástěnek, kalendářů a zpráv od učitelů. V navrženém systému byl žákovský rozcestník (dashboard) navržen zcela opačně. Přebírá roli aktivního filtru, který na úvodní stránce ukazuje výhradně sekci „Aktivní testy“. Žák není nucen prohledávat strukturu předmětů a složek, aby zjistil, co se od něj očekává; systém mu sám předloží pouze ty možnosti, které si v danou chvíli vyžadují jeho bezprostřední pozornost. Toto drastické snížení dostupných voleb okamžitě zkracuje reakční čas žáka a redukuje jeho předtestový stres.

Praktická aplikace Fittsova zákona [11]

Zatímco Hickův zákon řeší psychologii rozhodování, Fittsův zákon definuje limity lidské motoriky při cíleném pohybu. Zákon jasně deklaruje, že přesnost a rychlost interakce je závislá na vzdálenosti a především velikosti cílové plochy. Tento problém, v oblasti dotykových obrazovek známý jako „fat-finger problem“ (problém nepřesného dotyku prstem), představoval zásadní výzvu při optimalizaci aplikace pro mobilní telefony.

V testovacím rozhraní byla proto veškerá interakce optimalizována zvětšením aktivních dotykových ploch. Odpovědní volby u otázek typu výběr z možností (multiple/single choice) nejsou reprezentovány pouze malými, obtížně zasažitelnými zaškrťovacími políčky (radio button/checkbox). Namísto toho tvoří samotná odpověď širokou dotykovou kartu, která vyplňuje celou šířku obrazovky mobilního zařízení. Student tak může bezpečně klepnout kamkoliv do prostoru textu odpovědi, aniž by riskoval, že se překlikne na sousední možnost. Podobně byla velkoryse dimenzována i primární navigační tlačítka (např. „Odevzdat test“ nebo „Další otázka“), která jsou zároveň vždy jasně ohraničena barvou primární akce pro jejich okamžité rozpoznání.

7. Implementace frontendu

Úspěšná definice vizuálních pravidel a uživatelských cest představuje nezbytný teoretický rámec, na nějž bezprostředně navazuje samotná softwarová realizace. Přerod navržených prototypů do podoby produkčního kódu vyžaduje pevnou inženýrskou architekturu, která zajistí nejen vizuální věrnost návrhu, ale především stabilitu a okamžitou odezvu při interakci. Technologickým základem pro tuto transformaci se stal framework Flutter [16], jehož deklarativní přístup a práce s widgety přirozeně koresponduje s navrženým modulárním design systémem. Výchozím krokem pro udržitelnost celého řešení se proto stalo stanovení logické adresářové struktury, na kterou navazuje integrace robustního nástroje pro správu stavu a zajištění plynulé asynchronní komunikace se serverovou částí.

7.1. Softwarová architektura a struktura repozitáře

Základním kamenem udržitelnosti každého softwarového projektu je jeho adresářová struktura a míra modularity. Vzhledem k rozsahu a komplexitě vyvíjené aplikace by umístění veškerého zdrojového kódu do několika málo souborů vedlo k naprosté nepřehlednosti, špatné orientaci a vysokému riziku vzniku chyb při budoucím rozšiřování (tzv. anti-pattern *Spaghetti code*). Z tohoto důvodu byla ve frameworku Flutter aplikována striktní separace zodpovědností (Separation of Concerns).

Logické členění adresářové struktury

Zdrojový kód aplikace (adresář `lib/`) je rozdělen do několika specializovaných složek, z nichž každá plní jasně definovanou funkci:

- **pages/**: Obsahuje hlavní obrazovky aplikace (např. řídicí panel učitele, rozcestník studenta či editor testů). Tyto soubory fungují jako logické kontejnery, které skládají dohromady menší vizuální prvky a definují základní layout dané stránky.
- **components/**: Zde jsou umístěny znovupoužitelné stavební bloky uživatelského rozhraní. Patří sem například karty tříd (`class_card_widget.dart`), vyskakovací dialogy nebo barevné štítky určující typ otázky. Vyčlenění těchto prvků zaručuje, že jakákoliv vizuální úprava (např. změna stínu u karty) se automaticky a konzistentně propíše do celé aplikace.

- **layouts/**: Definuje globální obalovací struktury, které se opakují napříč obrazovkami, jako je například postranní menu (Sidebar) nebo horní navigační lišta.
- **theme/**: Izoluje veškerou vizuální logiku. Obsahuje definici použitých barev (`app_themes.dart`) a typografie. Tím je striktně oddělen vzhled od funkční logiky.
- **router/ a services/**: Složky vyhrazené pro konfiguraci cest aplikace a služby zajišťující komunikaci s externím API.
- **providers/**: Srdce aplikační logiky a správy stavu (State Management). Zde jsou definovány takzvané providery (s využitím balíčku Riverpod), které uchovávají a distribuují globální data napříč aplikací. Typickým příkladem je `auth_provider.dart`, který řídí stav přihlášení a oprávnění (zda je uživatel učitel, nebo student) nezávisle na vizuální vrstvě.
- **utils/**: Složka vyhrazena pro pomocné funkce a nástroje (utility), které přímo nesouvisí s vykreslováním uživatelského rozhraní, ale řeší specifické technické úkoly. Příkladem je logika pro stahování souborů či formátování textových řetězců.

Modularita kódu a izolace widgetů

Framework Flutter je ze své podstaty deklarativní a nutí vývojáře uvažovat v kontextu tzv. stromu widgetů (Widget Tree). Aby nedocházelo k vytváření monolitických a nepřehledných struktur, byla při vývoji důsledně aplikována kompozice widgetů.

Tento princip lze ilustrovat na příkladu editoru testů (`TestEditorWidget`). Namísto toho, aby jeden soubor obsahoval stovky řádků kódu definující grafiku, logiku přidávání otázek, nastavování počtu pokusů i výpočet časových limitů, byla funkcionality rozdělena do menších, specializovaných celků. Například logika pro nastavování času (okamžitý test vs. test s pevným termínem) byla vyčleněna do samostatného modulu `TimeSettingsWidget`. Hlavní formulář tak pouze "naslouchá" výsledkům tohoto modulu.

Tento modulární přístup přináší zásadní výhody. Zaprvé výrazně usnadňuje ladění kódu a hledání chyb, jelikož je každá funkcionality zapouzdřena ve vlastním souboru. Zadruhé snižuje zátěž frameworku při překreslování (re-rendrování) uživatelského rozhraní, protože při lokální změně stavu (např. posunutí přepínače u času) nedochází ke zbytečnému překreslování celého stromu obrovského formuláře, ale aktualizuje se pouze izolovaný widget.

7.2. Zpracování dat a komunikace s Backendem (API Client)

Zatímco vizuální vrstva zajišťuje interakci s uživatelem, skutečná data a obchodní logika sídlí na centrálním serveru. Architektura projektu proto vyžaduje spolehlivý most mezi frontendem a backendem, kterým proudí požadavky a odpovědi. Aby nedocházelo k duplikaci kódu a nepřehlednosti v jednotlivých vizuálních komponentách, byla veškerá síťová komunikace abstrahována do centralizované služby reprezentované třídou `ApiClient`.

Implementace třídy `ApiClient` a HTTP požadavky

Pro navazování spojení se serverem byl v rámci frameworku Flutter využit standardní balíček `http`. Třída `ApiClient` funguje jako obal (wrapper) nad tímto balíčkem a poskytuje unifikované metody pro základní CRUD operace, především GET (pro získávání dat, např. seznamu žáků) a POST (pro odesílání dat, např. vytvoření nového testu).

Zásadním přínosem této abstrakce je automatizované zpracování formátu JSON a centralizovaná správa autorizace. Jakmile se uživatel přihlásí, systém mu přidělí unikátní přístupový token. Třída `ApiClient` je navržena tak, že tento token automaticky vkládá do hlavičky každého odesílaného HTTP požadavku. Vývojář tak nemusí autorizaci řešit v každé obrazovce zvlášť, čímž se radikálně snižuje chybovost a zvyšuje bezpečnost komunikace.

Ošetření chyb a latence (Error Handling)

V reálném síťovém prostředí nelze spoléhat na to, že server odpoví vždy a okamžitě. Výpadek sítě nebo chyba na straně backendu by bez správného ošetření vedly k zamrznutí aplikace nebo neočekávanému pádu. Komunikační vrstva frontendu proto implementuje robustní zachytýný systém chybových stavů.

Pokud server vrátí chybový stavový kód (např. 401 Unauthorized nebo 500 Internal Server Error), `ApiClient` tuto skutečnost zachytí a vyvolá vlastní `ApiException`. Na úrovni uživatelského rozhraní se pak tento mechanismus projevuje dvěma způsoby:

- **Latence (čekání na data):** Během doby, kdy se čeká na odpověď serveru (tzv. asynchronní prodleva), rozhraní nemrzne, nýbrž zobrazuje indikátor načítání (`CircularProgressIndicator`). Uživatel je tak vizuálně informován o probíhajícím procesu.
- **Chyba spojení:** V případě selhání operace není aplikace ukončena, ale informaci z výjimky předá uživateli formou dočasné chybové notifikace v dolní části obrazovky (tzv. `SnackBar`), případně jej rovnou odhlásí, pokud vypršela platnost jeho tokenu.

Díky tomuto přístupu se podařilo vytvořit vysoce odolnou komunikační vrstvu, která naplňuje dříve definované UX heuristiky o viditelnosti stavu systému a prevenci chyb.

7.3. Správa stavu (State Management) a Riverpod

Vzhledem k reaktivní a deklarativní povaze Flutteru dochází k překreslování uživatelského rozhraní vždy, když se změní stav (např. uživatel vybere odpověď v testu nebo se dotáhnou data ze serveru). Nesprávná správa těchto stavových změn vede ke ztrátě dat, zbytečnému vytěžování procesoru neustálým překreslováním celého stromu a v konečném důsledku k „sekání“ aplikace.

Odklon od lokálního setState

V počátečních fázích vývoje lze pro jednoduché interakce (např. zaškrtnutí políčka) využít základní nativní funkci `setState`, kterou disponuje `StatefulWidget`. S rostoucí komplexitou aplikace se však tento přístup stává neudržitelným. Pokud potřebujeme například informaci o tom, zda je přihlášen učitel, nebo žák, sdílet mezi navigační lištou, hlavní stránkou a editorem testů, narazíme na problém tzv. *Prop Drillingu*. Tento termín označuje nežádoucí praxi, kdy se proměnné musí kaskádovitě předávat skrze konstruktory desítek widgetů, které s daty vůbec nepracují, jen je posílají hlouběji do stromu.

Z tohoto důvodu byl základní `setState` pro globální logiku opuštěn a architektura byla postavena na robustní knihovně **Riverpod**.

Architektura pomocí knihovny Riverpod

Riverpod (konkrétně jeho implementace přes `flutter_riverpod`) funguje jako reaktivní schránka pro stavy. Místo toho, aby stav (např. token uživatele nebo instance `ApiClient`) ležel uvnitř konkrétní obrazovky, je vyčleněn ven do tzv. `Provideru` (viz složka `providers/`).

Na úrovni uživatelského rozhraní jsou tradiční widgety nahrazeny speciálními typy, jako je `ConsumerWidget` nebo `ConsumerStatefulWidget`. Tyto widgety mají zabudovanou schopnost „naslouchat“ konkrétním `Providerům` z jakéhokoli místa v aplikaci.

Tento princip přináší aplikaci dvě kritické výhody:

- **Bezpečné čtení dat:** Libovolná komponenta (např. tlačítko „Odevzdat test“) si může bezprostředně vyžádat přístup k aktuálním datům uživatele pomocí příkazu `ref.read()`, aniž by musela složitě žádat o data své rodičovské prvky.
- **Optimalizace překreslování:** Pokud se změní stav v `Provideru`, Riverpod se postará o to, aby se v celém rozhraní překreslily pouze a jen ty widgety, které tento konkrétní stav opravdu využívají. Zbytek obrazovky zůstává nedotčen, čímž je zaručena naprostá plynulost animací i při složitých operacích s backendem.

Využití knihovny Riverpod tak vytvořilo pevnou, předvídatelnou a snadno testovatelnou datovou kostru celé aplikace.

7.4. Implementace klíčových a komplexních modulů

Abstraktní architektonická pravidla a nastavení komunikační vrstvy tvoří pouze základy aplikace. Pro ukázkou řešení komplexních problémů v prostředí frameworku Flutter byly vybrány dva stěžejní moduly: Editor testů pro pedagogy a Běh aktivního testu pro studenty.

Editor testů a dynamické banky otázek

Základním požadavkem na editor testů (`TestEditorWidget`) bylo umožnit učiteli plynule procházet databázi předem připravených otázek (Banky) a sestavit z nich výsledný test bez nutnosti přecházet na jiné obrazovky.

Algoritmus výběru funguje tak, že si v paměti obrazovky (*State*) udržuje vybraná ID otázek ve formě kolekce. Jakmile učitel rozklikne banku, formulář asynchronně stáhne její obsah a umožní otázky interaktivně přidávat. Složitost celého formuláře spočívá v závěrečné fázi odeslání. Formulář musí sesbírat data z několika nezávislých podmodulů – musí získat ID vybraných otázek, název testu, cílovou třídu a časová nastavení.

Běh testu

Prostředí pro řešení samotného testu studentem (`TestActiveWidget`) muselo být navrženo tak, aby bylo maximálně robustní proti chybám a výpadkům. Jakmile student test spustí, systém asynchronně stáhne pole s otázkami a uloží je do lokální mezipaměti.

Stavová logika této obrazovky hlídá dva hlavní parametry:

1. **Ukládání odpovědí:** Systém nesmí odesílat každou jednotlivou odpověď na server ihned po kliknutí. To by aplikaci neúměrně zpomalilo a zatížilo backend. Odpovědi se proto ukládají do lokální mapy (slovníku) v paměti zařízení. Díky využití deklarativního vykreslování se okamžitě barevně zvýrazní vybraná varianta, přičemž student má možnost mezi otázkami libovolně přeskakovat a volby měnit.
2. **Časovač a ochrana odevzdání:** Běh testu je řízen objektem `Timer`, který asynchronně odečítá zbývající sekundy. Logika rozhraní na tento odpočet reaktivně reaguje a v kritických posledních minutách varovně změní barvu ukazatele.

Jakmile je čas vyčerpán, nebo jakmile student klikne na odeslání, přichází na řadu algoritmus prevence chyb. Před samotným zabalením odpovědí do formátu JSON a jejich odesláním na server provede aplikace kontrolu dat. Pokud zjistí nezodpovězené otázky, vyvolá modální dialog. Po úspěšném odeslání (POST požadavek s výsledky) algoritmus studenta automaticky přesměruje z izolovaného režimu zpět a zobrazí mu formátované vyhodnocení.

7.5. Routování a ochrana přístupu (GoRouter)

Jelikož je výsledný projekt koncipován jako Single Page Application (SPA), veškerá navigace se odehrává čistě na straně klienta. Prohlížeč nenačítá ze serveru nové HTML dokumenty, ale framework dynamicky překresluje obsah na základě aktuální URL adresy. Aby byla správa těchto adres přehledná a plně odpovídala webovým standardům, byl pro navigaci použit balíček `go_router`.

Deklarativní navigace

Na rozdíl od staršího imperativního přístupu (`Navigator.push`) funguje `go_router` deklarativně. V centrálním souboru (`app_router.dart`) je definován ucelený strom všech povolených cest (např. `/teacher/classes` nebo `/student/test`). Každé URL adrese je pevně přiřazen konkrétní obalovací `Layout` a vnitřní `Page` widget. Tento přístup umožňuje snadné předávání parametrů v URL (např. `/student/test/123` pro test s ID 123) a zpřehledňuje celkovou strukturu aplikace.

Ochrana přístupu a přesměrování (Route Guards)

Nejkritičtějším úkolem routovacího systému je ovšem zajištění bezpečnosti a striktního oddělení rozhraní. Jak bylo definováno v analýze požadavků (kap. 5), student a učitel nesmí mít přístup do stejných částí systému.

Tento problém řeší implementace tzv. navigačních stráží (Route Guards) v parametru `redirect` uvnitř konfigurace routeru. Router je provázán se stavem aplikace (prostřednictvím `auth_provider.dart`) a při každém pokusu o změnu URL adresy vyhodnocuje následující pravidla:

1. **Autentizace:** Pokud se uživatel pokusí přistoupit na jakoukoliv interní stránku (např. Editor testů) a poskytovatel stavu neregistruje platný přihlašovací token, router navigaci okamžitě zablokuje a uživatele přesměruje zpět na přihlašovací obrazovku (`/login`).
2. **Autorizace a oddělení rolí:** Pokud je uživatel úspěšně přihlášen, router zkontroluje jeho roli. Jestliže se žák s platným tokenem pokusí manuálně zadat do prohlížeče adresu `/teacher/dashboard`, systém detekuje nepovolenou úroveň oprávnění. Rozhraní se nevykreslí a router uživatele bezpečně přesměruje na jeho domovskou žákovskou stránku.

Díky této logice je plně zaručena izolovanost pracovních prostředí. Z pohledu kódu se tak architektura nestará o to, aby v každém widgetu skrývala tlačítka před studentem, ale řeší problém elegantně a centralizovaně přímo na úrovni přístupu k dané URL adrese.

8. Diskuse a výsledky

V této kapitole jsou zhodnoceny dosažené výsledky z pohledu inženýrského návrhu, je definován plán nadcházejícího pilotního testování v školním prostředí a jsou diskutována technologická i designová omezení navrženého řešení.

Zhodnocení naplnění cílů a UX/UI principů

Z hlediska technologického a designového lze konstatovat, že primární cíle práce byly implementačně splněny. Využití frameworku Flutter a architektury Single Page Application (SPA) se ukázalo jako klíčové pro dosažení plynulosti a rychlosti odezvy, která stávajícím řešením často chybí. Důsledná aplikace Hickova zákona vedla k vytvoření striktně oddělených rozhraní pro pedagogy a studenty. Učitelský řídicí panel efektivně agreguje potřebná data a nástroj pro tvorbu testů zkracuje čas potřebný k jejich přípravě díky absenci zbytečného proklikávání a nepřehledných formulářů. Na straně studenta se podařilo úspěšně nasadit izolovaný testovací režim. Tím, že aplikace během testu skryje veškeré rušivé navigační prvky a nabízí vizuálně sjednocené dotykové plochy (reflektující Fittsův zákon), byla minimalizována vnější kognitivní zátěž.

Heuristické zhodnocení a plán pilotního testování

V současné době je frontendová část plně integrována s backendovým řešením a aplikace je úspěšně nasazena v testovacím prostředí. Vzhledem k harmonogramu vývoje a návaznosti na školní rok se reálné uživatelské testování se zapojením široké skupiny respondentů odehraje až v září s počátkem nového školního roku. Primární cílovou skupinou budou pedagogové a studenti z participujících gymnázií v Aši, Chebu a Jihlavě. Absence empirických dat z terénu v této fázi projektu je kompenzována heuristickým zhodnocením. Rozhraní bylo prozatím validováno proti Nielsenovým heuristikám (zejména prevence chyb a viditelnost stavu systému), což se projevilo například implementací varovných dialogů před odevzdáním nedokončeného testu nebo integrací lokální mezipaměti zabraňující ztrátě dat. Tím byla zajištěna vysoká výchozí úroveň použitelnosti ještě před nasazením do praxe.

Metodika budoucího testování:

Nasazení v praxi bude provázeno sběrem kvantitativních i kvalitativních dat.

- U pedagogů bude měřen průměrný čas potřebný k vytvoření testu s multimediálním obsahem a prostřednictvím dotazníků bude zjišťována míra jejich spokojenosti ve srovnání se staršími platformami.
- U studentů bude sledována plynulost vyplňování na různých typech mobilních zařízení a bude vyhodnocován vliv formativních prvků (okamžitá zpětná vazba, možnost oprav) na jejich studijní výsledky a předtestový stres opět prostřednictvím dotazníků.

Omezení navrženého řešení a technologické výzvy

Přestože zvolené technologie přinesly řadu výhod, implementace se neobešla bez určitých kompromisů a inženýrských výzev:

1. **Výkon na starších zařízeních (Flutter Web):** Ačkoliv je Flutter vynikající pro tvorbu konzistentního a responzivního UI bez spoléhání se na DOM, jeho webová exekuce může být na velmi starých zařízeních či slabých školních počítačích s neaktualizovanými prohlížeči náročnější na operační paměť a prvotní načítání aplikace. Během vývoje proto musela být věnována zvýšená pozornost optimalizaci překreslování pomocí knihovny Riverpod, aby nedocházelo k propadům výkonu při rychlé interakci.
2. **Omezení nativních prvků prohlížeče:** Jelikož Flutter Web pro zajištění maximální vizuální konzistence využívá pro vykreslování technologii HTML Canvas, obchází tím standardní DOM strom prohlížeče. Z uživatelského hlediska to představuje výzvu při interakcích, které uživatelé na běžném webu považují za samozřejmé – například označování textu kurzorem. Během vývoje se tento problém projevil například v přehledu třídy, kde si učitel nemůže přirozeně označit a zkopírovat e-mailové adresy studentů. Zpřístupnění tohoto chování vyžaduje explicitní využití speciálních obalovacích widgetů (např. `SelectableText`), což mírně zvyšuje vývojovou komplexitu.

Možnosti dalšího vývoje

Vytvořené uživatelské rozhraní slouží jako pevný a škálovatelný základ pro budoucí rozšiřování systému. Do dalších fází vývoje se nabízí implementace několika pokročilých modulů:

- **Real-time synchronizace a monitoring (Server-Sent Events):** Nahrazení současného dotazování (HTTP Polling) za technologii Server-Sent Events (SSE) by umožnilo učitelům sledovat postup žáků testem v reálném čase (např. indikovat, u jaké otázky se žáci nejvíce zasekávají) přímo na jejich řídicím panelu.
- **Rozšířená analytika (Dashboards):** Doplnění učitelského rozhraní o hlubší vizualizace historických dat a grafů úspěšnosti třídy, které by pedagogům pomohly snadněji identifikovat problematická místa ve výuce.

- **Nativní mobilní aplikace:** Díky využití multiplatformního frameworku Flutter je do budoucna možné s minimálním dodatečným úsilím zkompilovat existující kódovou základnu přímo do podoby nativních aplikací pro iOS a Android. To by studentům přineslo ještě plynulejší zážitek, nižší paměťovou náročnost a možnost plného využití nativních systémových push notifikací.
- **Propojení s externími aplikacemi:** Nabízí se i realizace API propojení se stávajícími školními informačními systémy (např. Bakaláři) za účelem automatické synchronizace hodnocení.

Závěrem lze konstatovat, že navržené a implementované řešení tvoří robustní funkční celek připravený pro produkční testování. Již nyní prokazatelně odstraňuje řadu architektonických UX bariér, se kterými se školy u stávajících platforem potýkají, a otevírá cestu k efektivnějšímu, modernějšímu a zejména formativnějšímu testování.

9. Závěr

Tato bakalářská práce se zaměřila na analýzu, návrh a implementaci uživatelského rozhraní (frontendu) pro moderní webovou aplikaci, jejímž primárním účelem je efektivní správa a tvorba didaktických testů na středních školách. Cíl práce bezprostředně vycházel z reálné potřeby gymnázií v Aši, Chebu a Jihlavě. V rámci analýzy stávajících platforem se totiž potvrdilo, že školám chybí vyvážený nástroj, který by poskytoval pokročilé didaktické možnosti a zachoval si naprosto čisté, minimalistické a intuitivní ovládání. Výsledkem této práce je ucelený a nasaditelný frontendový systém, který tyto protichůdné požadavky úspěšně propojuje.

Teoretická část práce shrnula klíčové pedagogické i psychologické principy, které formují kvalitní uživatelskou zkušenost (UX). Tento znalostní základ byl následně využit jako striktní mantinel pro návrh interakčních toků. Vzniklé rozhraní bylo logicky rozděleno do dvou nezávislých prostředí. Pro učitele byl navržen komplexní modulární řídicí panel a dynamický editor testů s podporou multimédií. Zcela opačný přístup byl zvolen pro prostředí žákovské, které se řídí principy mobile-first designu a v momentě spuštění testu přechází do tzv. izolovaného režimu, čímž maximalizuje soustředění a eliminuje rušivé elementy.

Praktická implementace prokázala, že framework Flutter ve spojení s architekturou Single Page Application (SPA) je pro vývoj vzdělávacího softwaru vhodný, a to především díky okamžité plynulosti interakce a striktnímu dodržení pravidel Material Designu.

V současné fázi projektu je frontendová část plně integrována s backendovým řešením a aplikace je nasazena v testovacím prostředí. Tento provozní stav představuje robustní základ připravený pro plošný pilotní provoz, který bude na participujících gymnáziích spuštěn se začátkem nového školního roku. Očekávaná zpětná vazba od pedagogů a studentů bude následně sloužit jako primární podklad pro další iterativní ladění a případnou realizaci pokročilých nadstaveb, jakými by mohly být nativní mobilní aplikace či hlubší datová analytika.

Vzniklý frontend nejen efektivně řeší počáteční zadání, ale díky hlubokému zaměření na psychologii uživatelského rozhraní posouvá úroveň digitálního testování na středních školách na vyšší, intuitivnější a kognitivně přívětivější úroveň.

Seznam použitých zdrojů

1. JUNKOVÁ, Jana. *DIDAKTICKÉ TESTOVÁNÍ*. 2009. Dostupné také z: https://is.muni.cz/el/1441/podzim2009/ZS1BK_PDD/didakticke_testovani.pdf. [cit. 2026-05-07].
2. SHUTE, Valerie J.; BECKER, Betsy Jane. *Innovative Assessment for the 21st Century*. 2010. Dostupné také z: <https://download.e-bookshelf.de/download/0000/0065/59/L-G-0000006559-0002369485.pdf>. [cit. 2026-05-07].
3. ROEDIGER III, Henry L.; KARPICKE, Jeffrey D. *The Power of Testing Memory*. 2006. Dostupné také z: http://psychnet.wustl.edu/memory/wp-content/uploads/2018/04/Roediger-Karpicke-2006_PPS.pdf. [cit. 2026-05-07].
4. HATTIE, John; TIMPERLEY, Helen. *The Power of Feedback*. 2007. Dostupné také z: <https://simvilledev.ku.edu/sites/default/files/PD%20Resources/Hattie%20power%20of%20feedback%5B1%5D.pdf>. [cit. 2026-05-07].
5. OKONKWO, Charles. *Assessment of User Experience (UX) Design Trends in Mobile Applications*. 2024. Dostupné také z: https://www.researchgate.net/publication/382804453_Assessment_of_User_Experience_UX_Design_Trends_in_Mobile_Applications. [cit. 2026-05-07].
6. ALKHADHER, Othman; ANDERSON, Neil Robert; CLARKE, David. *Computer-based testing: A review of recent developments in research and practice*. 1994. Dostupné také z: https://www.researchgate.net/publication/247515840_Computer-based_testing_A_review_of_recent_developments_in_research_and_practice. [cit. 2026-05-07].
7. ZAHARIAS, Panagiotis; POULYMENAKOU, Angeliki. *Developing a Usability Evaluation Method for e-Learning Applications: Beyond Functional Usability*. 2009. Dostupné také z: https://www.researchgate.net/publication/220302590_Developing_a_Usability_Evaluation_Method_for_e-Learning_Applications_Beyond_Functional_Usability. [cit. 2026-05-07].
8. GEORGIEV, Tsvetozar Stefanov; GEORGIEVA, Evgenia. *User Interface Design for Mobile Learning Applications*. 2009. Dostupné také z: https://www.researchgate.net/publication/274379765_User_Interface_Design_for_Mobile_Learning_Applications. [cit. 2026-05-10].
9. NIELSEN, Jakob. *10 Usability Heuristics for User Interface Design*. 1994. Dostupné také z: <https://www.nngroup.com/articles/ten-usability-heuristics/>. [cit. 2026-05-10].

10. ROSATI, Luca. *How to design interfaces for choice: Hick-Hyman law and classification for information architecture*. 2013. Dostupné také z: <https://www.lucarosati.it/download/papers/hick-law-classification.pdf>. [cit. 2026-05-10].
11. MACKENZIE, Scott. *Fitts' Law*. 2018. Dostupné také z: <https://www.yorku.ca/mack/hhci2018.pdf>. [cit. 2026-05-10].
12. PATEL, Prakash. *A Guide to Material Design, a Modern Software Design Language*. 2016. Dostupné také z: https://www.researchgate.net/publication/301655519_A_Guide_to_Material_Design_a_Modern_Software_Design_Language. [cit. 2026-05-10].
13. MIKA, Alex; VASYLENKO, Juri. *SPA vs. MPA Comparison*. 2026. Dostupné také z: <https://www.ramotion.com/blog/spa-vs-mpa/>. [cit. 2026-05-18].
14. RED HAT. *What is a REST API?* 2020. Dostupné také z: <https://www.redhat.com/en/topics/api/what-is-a-rest-api>. [cit. 2026-05-18].
15. JAIN, Vivek. *A COMPARATIVE ANALYSIS OF SINGLE PAGE APPLICATIONS (SPAS) AND MULTI PAGE APPLICATIONS (MPAS)*. 2022. Dostupné také z: https://www.researchgate.net/publication/390112322_A_COMPARATIVE_ANALYSIS_OF_SINGLE_PAGE_APPLICATIONS_SPAS_AND_MULTI_PAGE_APPLICATIONS_MPAS. [cit. 2026-05-18].
16. GOOGLE. *Flutter architectural overview*. 2026. Dostupné také z: <https://docs.flutter.dev/resources/architectural-overview>. [cit. 2026-05-18].
17. GOOGLE. *Web support for Flutter*. 2026. Dostupné také z: <https://docs.flutter.dev/platform-integration/web>. [cit. 2026-05-18].
18. GOOGLE. *Dart overview*. 2026. Dostupné také z: <https://dart.dev/overview>. [cit. 2026-05-18].
19. APIGEE. *Web API Design: The Missing Link*. 2018. Dostupné také z: <https://cloud.google.com/files/apigee/apigee-web-api-design-the-missing-link-ebook.pdf>. [cit. 2026-05-18].
20. ROUSSELET, REMI. *Riverpod framework*. 2026. Dostupné také z: <https://riverpod.dev>. [cit. 2026-05-18].
21. FLUTTERFLOW. *FlutterFlow*. 2026. Dostupné také z: <https://flutterflow.io>. [cit. 2026-05-18].

Seznam obrázků

5.1. Návrh hlavního řídicího panelu učitele s využitím modulárního systému karet pro minimalizaci kognitivní zátěže.	40
5.2. Detailní analytika konkrétní třídy s barevnou signalizací prioritních úkolů a přehledem aktivních testů.	41
5.3. Intuitivní editor testových úloh s dominantní drag-and-drop zónou pro multimédia a funkcí náhledu z perspektivy žáka.	42
5.4. Konfigurační rozhraní pro sestavení testu s přepínači definujícími parametry didaktického cyklu a formativní zpětné vazby.	43
5.5. Minimalistický rozcestník žáka. Pozornost je primárně směřována na sekci aktivních testů s blížícím se termínem odevzdání.	43
5.6. Detail konkrétního předmětu poskytující žákovi okamžitý přehled o historii splněných testů a dosažených výsledcích.	44
5.7. Izolovaný testovací režim. Rozhraní eliminuje vizuální smog a využívá široké dotykové karty pro minimalizaci problému nepřesných dotyků na mobilním zařízení.	44
5.8. Rozhraní pedagoga pro kontrolu testů. Návrh kombinuje automaticky vyhodnocené úlohy s prostorem pro manuální zápis bodů a individuální formativní zpětné vazby k otevřeným otázkám.	45

A. Externí přílohy

Prvotní UI návrh aplikace vytvořený v FlutterFlow je dostupný na adrese:

<https://bak-jirotko.flutterflow.app>

Externí přílohy této bakalářské práce jsou dostupné v repozitáři na adrese:

<https://github.com/Jirotkins/BP-Jirotko-Heger>.

Repozitář obsahuje zdrojové kódy frontendové i backendové části, inicializační databázové skripty a konfiguraci pro lokální spuštění pomocí Dockeru. Tabulka A.1 uvádí přehled základní struktury a význam jednotlivých částí se zaměřením na klientskou část.

Cesta	Popis
README.md	Stručný popis repozitáře.
docker-compose.yml	Konfigurační soubor pro snadné lokální spuštění celého aplikačního stacku v kontejnerech.
*.sql	Skripty pro inicializaci databázového schématu a naplnění mock daty.
backend/	Zdrojové kódy serverové části, která je primárním výstupem souběžné bakalářské práce kolegy Adama Hegera.
frontend/	Kořenový adresář frontendové webové aplikace, která je primárním výstupem této bakalářské práce.
frontend/lib/main.dart	Vstupní bod klientské aplikace.
frontend/lib/pages/	Hlavní obrazovky aplikace rozdělené do logických celků (např. řídicí panel učitele, rozcestník žáka).
frontend/lib/components/	Znovupoužitelné stavební bloky uživatelského rozhraní (tlačítka, modální dialogy).
frontend/lib/providers/	Aplikační logika a reaktivní správa stavu.
frontend/lib/router/	Konfigurace cest pro bezpečné směrování v aplikaci (GoRouter).
frontend/lib/layouts/	Globální obalovací struktury (např. boční menu).
frontend/lib/services/	Třídy a metody zajišťující asynchronní komunikaci s backendovým API.
frontend/lib/utils/	Pomocné funkce, formátovače a sdílené nástroje nezávislé na vizuální vrstvě.
frontend/lib/theme/	Definice globálního vizuálního stylu aplikace (barvy, typografie).

Tabulka A.1.: Struktura repozitáře s externími přílohami